Implementacja systemu informatycznego wspierającego proces online tutoringu

(Development and Implementation of an
Online Tutoring Support System)

Radosław Madzia

Praca inżynierska

Promotor: dr Wiktor Zychla

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

31 sierpnia 2026

Streszczenie

Praca przedstawia projekt oraz implementację szkieletu systemu informatycznego wspierającego proces online tutoringu poprzez integrację aplikacji desktopowej z platformą webową. Celem systemu jest umożliwienie przechwytywania danych obrazu oraz dźwięku, zapisywanie materiałów oraz ich udostępnianie za pośrednictwem aplikacji webowej. System składa się z dwóch części: aplikacji desktopowej odpowiedzialnej za przechwytywanie, przetwarzanie oraz transfer materiałów multimedialnych oraz webowej, realizującej funkcjonalność przechowywania, udostępniania i odtwarzania zamieszczonych materiałów. Aplikację desktopową wytworzono przy użyciu języka Rust oraz bibliotek min.: Slint, FFmpeg, Cpal oraz scap. Aplikacja webowa oparta została na technologii ASP.NET Core Razor Pages z wykorzystaniem ASP.NET Core Identity, OpenIddict oraz Entity Framework Core.

The purpose of this study was both to design and implement a software system framework supporting the online tutoring process through the integration of a desktop application with a web platform. The aim of the system is to enable the capture of video and audio data, the storage of recorded materials, and their sharing through a web application. We can define two main parts of the system: a desktop application responsible for capturing, processing, and transferring multimedia files, and a web application providing functionality for storing, sharing, and playing back the uploaded materials. The desktop application was developed using the Rust programming language and libraries including Slint, FFmpeg, CPAL, and scap. The web application is based on ASP.NET Core Razor Pages and utilizes ASP.NET Core Identity, OpenIddict, and Entity Framework Core.

Spis treści

1. Wprowadzenie **9**

1.1. Przedstawienie problematyki 9

1.2. Cel i zakres pracy 10

2. Analiza istniejących rozwiązań **13**

2.1. Charakterystyka istniejących rozwiązań 13

2.1.1. Systemy nastawione na udostępnianie materiałów 13

2.1.2. Aplikacje i systemy nastawione na rejestrowanie obrazu . . . 14

2.2. Porównanie metodyk obsługi plików 16

2.3. Najbardziej istotne funkcjonalności 18

2.4. Wnioski z analizy 20

3. Analiza problemu i wymagań systemu **23**

3.1. Kontekst projektowanego systemu 23

3.2. Analiza użytkowników systemu 25

3.2.1. Historyjki użytkowników 26

3.3. Wymagania funkcjonalne 27

3.4. Wymagania niefunkcjonalne 29

3.5. Ograniczenia projektowe 31

4. Projekt systemu **33**

4.1. Architektura systemu 33

4.1.1. Zarys ogólny architektury 33

4.1.2. Diagram kontekstu systemu – C4 System Context 34

4.1.3. Diagram kontenerów – C4 Container 34
4.1.4. Diagram komponentów – C4 Component 35
4.2. Model Danych 37
4.2.1. Struktura modelu danych 38
4.3. Proces Transferu Danych 40
4.3.1. Transfer fragmentowy 41
4.4. Proces uwierzytelniania użytkownika 42
4.4.1. Przebieg procesu uwierzytelniania 43

5. Implementacja systemu 45
5.1. Wybrane narzędzia oraz technologie 46
5.1.1. Aplikacja webowa 46
5.1.2. Aplikacja desktopowa 47
5.2. Implementacja potoku przechwytywania i przetwarzania obrazu . . . 48
5.2.1. Przechwytywanie klatek obrazu 48
5.2.2. Przygotowanie klatek do enkodowania 49
5.2.3. Kodowanie strumienia video 50
5.3. Implementacja potoku przetwarzania dźwięku 50
5.3.1. Buforowanie sampli audio 51
5.3.2. Resampling sygnału 53
5.3.3. Miksowanie sampli 53
5.3.4. Enkodowanie dźwięku 55
5.4. Multipleksowanie, synchronizacja i zapis do pliku 55
5.4.1. Znaczniki czasowe i synchronizacja 55
5.4.2. Zakończenie zapisu 56
5.5. Współbieżne przetwarzanie strumieni 56
5.5.1. Organizacja współbieżnego potoku 57
5.5.2. Komunikacja pomiędzy potokami 57
5.5.3. Problem różnicy szybkości przetwarzania na przykładzie wideo 58
5.5.4. Buforowanie i utrata danych 58

5.5.5. Ograniczenia programowego enkodowania obrazu 59
5.6. Implementacja przesyłania plików 59
5.6.1. Upload dzielony na fragmenty 59
5.6.2. Przechowywanie pliku tymczasowego 60
5.6.3. Kontrola stanu uploadu 60
5.7. Rozszerzalność implementacji 60

6. Część dla programisty 63
6.1. Przygotowanie środowiska i kompilacja 64
6.1.1. Aplikacja webowa 64
6.1.2. Aplikacja desktopowa 65
6.2. Metryka kodu 66

7. Instrukcja użytkownika 67
7.1. Zmiana podstawowych parametrów przechwytywania obrazu 67
7.2. Wybór źródeł przechwytywanych danych i rozpoczęcie nagrywania . 67
7.3. Przeglądanie plików dostępnych w aplikacji webowej 68
7.4. Transfer przechwyconych plików 68

8. Podsumowanie i dalszy rozwój projektu 73
8.1. Kierunki dalszego rozwoju 73

Bibliografia 75

Rozdział 1.

Wprowadzenie

Na przestrzeni ostatnich lat platformy umożliwiające prowadzenie zajęć, spotkań oraz konsultacji online stały się nieodzownym elementem procesu kształcenia. Rozwój nauczania zdalnego oraz upowszechnienie pracy i komunikacji za pośrednictwem internetu sprawiły, że rozwiązania projektowane przede wszystkim z myślą o dużych organizacjach i przedsiębiorstwach zaczęły być powszechnie wykorzystywane również przez indywidualnych użytkowników. W rezultacie powstał szeroki rynek narzędzi wspierających komunikację w czasie rzeczywistym, prowadzenie zajęć oraz tworzenie i udostępnianie materiałów dydaktycznych.

1.1. Przedstawienie problematyki

Wśród dostępnych i powszechnie rozpoznawanych rozwiązań można wyróżnić między innymi platformy przeznaczone do prowadzenia wideokonferencji, takie jak: *Microsoft Teams*[1], *Zoom*[2] , *Google Meet*[3], *Cisco Webex*[4], *Discord*[5] czy *Proton Meet*[6]. Oferują one funkcje pozwalające na komunikację audio-wideo, udostępnianie ekranu oraz, w zależności od platformy i konfiguracji konta, rejestrowanie przebiegu spotkania. Przykładowo, *Microsoft Teams* umożliwia rejestrowanie spotkań wraz z dźwiękiem, obrazem oraz udostępnioną zawartością ekranu, które są następnie przechowywane w usługach opartych na chmurze powiązanych z organizacją.

Obok rozwiązań umożliwiających prowadzenie spotkań funkcjonują również narzędzia skoncentrowane na samym tworzeniu materiałów wideo. Wspaniałym przykładem jest *Loom*[7], który pozwala rejestrować obraz ekranu, kamerę oraz dźwięk zarówno bezpośrednio z poziomu przeglądarki (między innymi za pomocą rozszerzenia dla przeglądarki Chrome) jak i aplikacji desktopowej. Nagrane materiały są następnie automatycznie umieszczane w bibliotece usługi i mogą być udostępniane za pomocą wygenerowanego odnośnika. Ten model pokazuje, że środowisko przeglądarki jest wystarczające do realizacji przedstawionego celu w sekcji 1.2..

Warto zauważyć, że istotną cechą wspólną większości wyżej wymienionych plat-

form jest udostępnianie funkcjonalności zarówno za pośrednictwem aplikacji desktopowej oraz przeglądarki internetowej. Zatem można wnioskować, że różnorodność udostępnionych w ramach jednego systemu rozwiązań pozwala dopasować sposób wykorzystania do charakteru wykonywanego przez użytkownika zadania.

Kolejną grupę stanowią programy przeznaczone głównie do lokalnego użytku. Ich reprezentantami są min. *OBS Studio[8], vMix [9], Meld[10]* oraz *BandiCam*[11]. Narzędzia tego rodzaju oferują znacznie większą kontrolę nad procesem przechwytywania, edycji, enkodowania i zapisywania materiału. Ich podstawowym zastosowaniem jest jednak tworzenie pliku wideo na urządzeniu użytkownika lub przesłanie obrazu w czasie rzeczywistym do zewnętrznego serwisu streamingowego.

Przedstawione rozwiązania pokazują, że funkcjonalności związane z procesem tworzenia materiałów wideo, komunikacją w czasie rzeczywistym oraz przechowywaniem plików wynikowych są obecnie dostępne w wielu dojrzałych narzędziach i systemach. Poszczególne platformy realizują jednak te zadania w odmienny sposób i koncentrują się na różnych elementach całego procesu. *Microsoft Teams, Zoom* czy *Google Meet* integrują rejestrowanie spotkań z platformą komunikacyjną i infrastrukturą chmurową, natomiast narzędzia takie jak *OBS Studio* skupiają się na lokalnym przechwytywaniu, enkodowaniu i transmisji materiału. Z kolei aplikacje internetowe mogą wykorzystywać dostępne w przeglądarkach interfejsy programistyczne, takie jak *Screen Capture API*[12] będącym rozszerzeniem *Media Capture and Streams API*[13], do uzyskania dostępu do obrazu ekranu oraz wybranych źródeł dźwięku.

1.2. Cel i zakres pracy

W ramach niniejszej pracy przeprowadzono analizę istniejących rozwiązań, wymagań funkcjonalnych i niefunkcjonalnych stawianych przed takimi systemami oraz podjęto próbę ich połączenia poprzez zaprojektowanie i implementację wybranych mechanizmów w szkielet spójnego systemu, który umożliwi dalsze prace rozwojowe.

Zaimplementowany system składa się z dwóch niezależnych, komunikujących się ze sobą części: aplikacji desktopowej oraz aplikacji webowej. Aplikacja desktopowa odpowiada za przechwytywanie obrazu wraz ze ścieżką dźwiękową, enkodowanie i multipleksing danych do materiału wideo oraz za transfer wybranych przez użytkownika plików wynikowych na serwer. Natomiast aplikacja webowa zapewnia mechanizmy uwierzytelniania użytkownika, przechowywania materiałów, zarządzania oraz późniejszego ich odtwarzania. Szczególną uwagę poświęcono mechanizmom przechwytywania danych z urządzeń wyjściowych wideo i wejściowych audio, generowaniu plików wynikowych oraz komunikacji i przesyłowi pomiędzy obiema częściami systemu. Implementacja oraz pre release build dostępne są w prywatnym repozytorium pod adresem: `https://github.com/MADkarmi/ScreenCap`. By uzyskać do niego dostęp, należy przesłać prośbę do autora pracy.

Motywacją do realizacji pracy była chęć praktycznego zapoznania się z trudnościami wynikającymi z budowy systemu łączącego tak różnorodne zagadnienia. Implementacja rozwiązania niejako od podstaw stanowi intrygującą tematykę z zakresu inżynierii oprogramowania, wymagającą wykorzystania szerokiego zakresu wiedzy z obszarów wytwarzania aplikacji desktopowych jak i webowych, przetwarzania obrazu i dźwięku, komunikacji sieciowej, uwierzytelniania, zarządzania danymi oraz przetwarzania i przechowywania obiektów *BLOB*[14]. Istotnym zatem elementem realizacji pracy jest możliwość poznania tych mechanizmów nie tylko na poziomie użytkowym, lecz poprzez praktyczną implementację i integrację w ramach jednolitego systemu

Rozdział 2.

Analiza istniejących rozwiązań

2.1. Charakterystyka istniejących rozwiązań

2.1.1. Systemy nastawione na udostępnianie materiałów

Wśród istniejących rozwiązań skupionych wokół tworzenia i/lub udostępniania materiałów wideo można wyróżnić trzy podstawowe grupy platform, różniących się przede wszystkim zakresem realizowanych funkcji oraz miejscem procesu tworzenia materiału w całym modelu działania systemu:

1. platformy skoncentrowane na przechowywaniu, publikowaniu i udostępnianiu gotowych treści.
2. systemy, których głównym przeznaczeniem jest tworzenie, przechowywanie oraz udostępnianie gotowych materiałów wideo.
3. platformy komunikacyjne, w których możliwość rejestrowania i późniejszego udostępniania nagrań stanowi jedną z funkcjonalności towarzyszących prowadzeniu spotkań lub zajęć online.

Platformy grupy pierwszej koncentrują się przede wszystkim na obsłudze gotowych materiałów wideo. Najpopularniejszymi są *YouTube*[15] czy *Dailymotion*[17]. Ich podstawowe funkcje obejmują przesyłanie plików na serwer, ich przechowywanie, przetwarzanie, odtwarzanie oraz udostępnianie innym użytkownikom. Proces rejestrowania obrazu nie stanowi przy tym integralnego elementu systemu. Materiał musi zostać przygotowany wcześniej, przy użyciu zewnętrznego narzędzia lub urządzenia, a następnie przesłany do platformy. Rozdzielenie procesu tworzenia i publikowania materiału pozwala na wykorzystanie dowolnego oprogramowania przeznaczonego do rejestrowania i edycji obrazu.

Odmienne podejście prezentują reprezentanci grupy drugiej tacy jak: *Loom*[7] czy też *Vimeo*[16], których funkcjonalności obejmują zarówno rejestrowanie obrazu

ekranu, obrazu z kamery oraz dźwięku, jak i późniejsze przechowywanie oraz udostępnianie utworzonych materiałów. Nagrane materiały są automatycznie umieszczane w bibliotece użytkownika i mogą być następnie odtwarzane oraz udostępniane za pomocą odnośnika. W przypadku tej grupy proces tworzenia materiału i jego dystrybucji został zintegrowany w ramach jednego systemu.

Trzeci model reprezentują platformy przeznaczone przede wszystkim do prowadzenia komunikacji w czasie rzeczywistym, takie jak *Microsoft Teams*, *Zoom* czy *Google Meet*. W ich przypadku nagranie może powstać jako rezultat prowadzonego spotkania, zajęć lub konsultacji. Rejestrowanie obrazu i dźwięku nie jest zatem głównym celem systemu, lecz funkcją uzupełniającą podstawowe mechanizmy komunikacji. W zależności od platformy i konfiguracji konta powstałe nagrania mogą być następnie przechowywane w infrastrukturze powiązanej z usługą i udostępniane uczestnikom spotkania.

Podział ten pozwala jednoznacznie rozróżnić systemy, w których materiał wideo jest głównym przedmiotem działania użytkownika, od rozwiązań, w których powstaje on jako rezultat prowadzonej komunikacji. Oczywiście każdy z systemów spośród powyżej wymienionych grup implementuje metody zarządzania dostępu do treści przez użytkowników.

2.1.2. Aplikacje i systemy nastawione na rejestrowanie obrazu

Drugą grupę analizowanych rozwiązań stanowią aplikacje przeznaczone przede wszystkim do lokalnego rejestrowania obrazu i dźwięku oraz generowania na ich podstawie plików wideo. W przeciwieństwie do wcześniej opisanych platform, których istotnym elementem jest przechowywanie i udostępnianie gotowych materiałów, programy tego rodzaju koncentrują się na procesie pozyskiwania danych z urządzenia użytkownika, ich przetwarzaniu oraz zapisywaniu wyniku w postaci pliku multimedialnego.

Jednym z najbardziej rozpoznawalnych przykładów tego rodzaju oprogramowania jest opensource'owy *OBS Studio*[8]. Program w głównej mierze [18] stworzony z wykorzystaniem języków *C* oraz *C++* umożliwia przechwytywanie całego ekranu, wybranych okien aplikacji oraz innych dostępnych źródeł obrazu. Pozwala również na jednoczesne wykorzystanie wielu źródeł dźwięku, w tym urządzeń wejściowych oraz wyjściowych. Poszczególne źródła mogą być łączone w ramach scen, co pozwala użytkownikowi określić sposób komponowania wynikowego pliku. Dodatkowo *OBS Studio* udostępnia funkcjonalność broadcastowania live streamingu na skonfigurowaną przez użytkownika platformę.

Istotną cechą *OBS Studio* jest możliwość szczegółowej konfiguracji procesu zapisu - użytkownik może określać min:. rozdzielczość, liczbę klatek na sekundę, parametry enkodowania oraz format zapisywanego pliku. Oprogramowanie wykorzystuje enkodery softwarowe oraz akceleracje sprzętową, dzięki czemu przetwarzanie mate-

riału może zostać dostosowane do możliwości wykorzystywanego urządzenia.

Podobny model działania reprezentuje *BandiCam*[11] - aplikacja przeznaczona do rejestrowania obrazu ekranu, określonych obszarów pulpitu oraz danych z urządzeń wejściowych. Oprócz przechwytywania, umożliwia konfigurację parametrów nagrywania oraz formatu zapisu, niestety jest okrojona względem *OBS Studio* o funkcjonalność streamingu do platform zewnętrznych.

Rozwiązania tej grupy zapewniają znacznie większą kontrolę nad procesem rejestrowania obrazu oraz dźwięku względem systemów, w których nagrywanie stanowi dodatkową funkcję. Użytkownik może bezpośrednio określić źródła obrazu, dźwięku, parametry oraz sposób enkodowania przechwyconych danych. Wiąże się to jednak z koniecznością realizacji całości procesu, w tym przechwytywania, synchronizacji, enkodowania i multipleksingu, po stronie urządzenia użytkownika, co nakłada spory narzut obliczeniowy. Zatem to na użytowniku spoczywa odpowiedzialność, za właściwy dobór parametrów, by hardware, na którym pracuje, nie został przeciażony.

Warto również zwrócić uwagę na różnicę między lokalnym rejestrowaniem, a przechowywaniem materiałów w ramach platform internetowych. W przypadku aplikacji desktopowych, końcowym rezultatem działania programu jest zazwyczaj plik znajdujący się na lokalnym urządzeniu. Zatem udostępnienie materiału innym użytkownikom wymaga dokonania dodatkowych operacji przesłania do wybranej usługi. Proces ten może być przeprowadzony manualnie, bądź automatycznie w zależności od tego, czy aplikacja została zintegorowana z serwerem obsługującym przechowywanie matriałów.

Warto zauważyć, że rozdzielenie rejestrowania i udostępniania materiałów pozwala na niezależne realizowanie obu funkcjonalności - aplikacja odpowiedzialna za przechwytywanie może ograniczyć się do zapewnienia odpowiedniej jakości oraz wydajności procesu rejestrowania, natomiast część serwerowa systemu może zajmować się przechowywaniem, zarządzaniem i udostępnianiem gotowych plików. Taki podział odpowiedzialności wydaje się być naturalny i występuje w projektowanym systemie - aplikacji desktopowej powierzony został proces przechwytywania danych z urządzeń wejściowych i tworzenia materiału wynikowego, oraz jako osobny proces - upload plików na serwer, a dalsza obsługa realizowana jest przez alikację webową.

Analiza tej grupy rozwiązań prezentuje jednoznaczny obraz istotnych elementów tego procesu:

- dostęp do urządzeń wejścia i wyjścia odpowiedzialnych za obsługę audio oraz wideo
- przechwytywanie danych z wyżej wymienionych urządzeń bez wpływu na działanie systemu operacyjnego
- synchronizacja danych w czasie
- enkodowanie oraz multipleksing do pliku wynikowego

Zagadnienia te stanowią podstawę technicznej części projektowanego rozwiązania.

2.2. Porównanie metodyk obsługi plików

Sposób obsługi materiału wideo zależy nie tylko od funkcjonalności oferowanych przez daną platformę, lecz również od podziału odpowiedzialności pomiędzy urządzeniem użytkownika a infrastrukturą serwerową. W analizowanych rozwiązaniach można wyróżnić trzy podstawowe metodyki obsługi plików:

- lokalną,
- opartą na przesyłaniu gotowego materiału do zewnętrznej platformy,
- zintegrowaną, w której proces tworzenia, przechowywania i udostępniania materiału realizowany jest w ramach jednego systemu.

Pierwsza z wyróżnionych metodyk zakłada lokalną obsługę materiału. W tym modelu aplikacja działająca na urządzeniu użytkownika odpowiada za przechwycenie i przetworzenie danych oraz zapisanie gotowego materiału w systemie plików. Przykładem takiego podejścia jest *OBS Studio*[8]. Utworzony plik pozostaje na urządzeniu użytkownika i może być następnie wykorzystany przez inne aplikacje lub przesłany do wybranej usługi. Model ten zapewnia użytkownikowi dużą kontrolę nad procesem tworzenia materiału, jednak funkcje związane z jego późniejszym przechowywaniem i udostępnianiem wymagają wykorzystania dodatkowych mechanizmów lub usług.

Drugą metodykę stanowi przesył gotowego materiału do platformy odpowiedzialnej za udostępnianie treści. W tym przypadku proces tworzenia pliku odbywa się niezależnie od systemu odpowiedzialnego za jego dalszą obsługę. Użytkownik przygotowuje materiał stosując wybrane oprogramowanie, a następnie przesyła plik wynikowy do usługi, która przejmuje odpowiedzialność za jego przechowywanie, przetwarzanie, odtwarzanie oraz udostępnianie. Model ten jest charakterystyczny dla platform takich jak *YouTube*[15] czy *Dailymotion*[17]. Rozdzielenie obu etapów pozwala użytkownikowi korzystać z dowolnych narzędzi do tworzenia materiałów, jednocześnie zapewniając dostęp do rozbudowanej infrastruktury przeznaczonej do ich dalszej obsługi.

W przypadku tego modelu istotne znaczenie ma sam proces przekazania pliku pomiędzy urządzeniem użytkownika a serwerem. Po utworzeniu materiał musi zostać przekazany do infrastruktury platformy, a sam proces jest zależny od czasu trwania oraz poprawności komunikacji. Znaczenie tego etapu wzrasta wraz z rozmiarem danych, dlatego platformy obsługujące duże pliki muszą uwzględniać między innymi możliwość wystąpienia przerw w transmisji oraz konieczność zapewnienia odpowiedniego mechanizmu kontroli procesu przesyłania.

Trzecią metodykę stanowi zintegrowana obsługa materiału. W tym przypadku proces tworzenia materiału oraz jego dalsze przechowywanie i udostępnianie są elementami jednego ekosystemu. Użytkownik może rozpocząć rejestrowanie materiału za pomocą aplikacji lub interfejsu udostępnianego przez platformę, a po zakończeniu procesu materiał zostaje przekazany do infrastruktury usługi bez konieczności wykonywania przez użytkownika osobnej operacji przesyłania. Przykładem takiego rozwiązania jest *Loom*[7], który integruje funkcje rejestrowania obrazu i dźwięku z biblioteką materiałów oraz mechanizmami ich udostępniania.

Podobny sposób integracji można zaobserwować w platformach komunikacyjnych, takich jak *Microsoft Teams*[1], *Zoom*[2] czy *Google Meet*[3]. W tym przypadku materiał jest tworzony w ramach prowadzonego spotkania, a po jego zakończeniu może zostać zapisany w infrastrukturze powiązanej z daną usługą. W odróżnieniu od *Loom* głównym celem tych platform nie jest jednak tworzenie i udostępnianie materiałów wideo, lecz komunikacja w czasie rzeczywistym. Nagrywanie stanowi więc funkcję dodatkową, zintegrowaną z podstawowym procesem prowadzenia spotkania.

Tabela 2.1: Zestawienie metodyk obsługi plików

Etap obsługi materiału	**Obsługa lokalna**	**Przesłanie gotowego materiału**	**Obsługa zintegrowana**
Utworzenie materiału	aplikacja lokalna	aplikacja zewnętrzna	platforma
Przetwarzanie materiału	urządzenie użytkownika	zależnie od rozwiązania	platforma i/lub urządzenie
Początkowe przechowywanie	urządzenie użytkownika	urządzenie użytkownika	infrastruktura platformy
Przesłanie na serwer	opcjonalne	wymagane	automatyczne lub zintegrowane
Długoterminowe przechowywanie	lokalne	serwer platformy	serwer platformy
Odtwarzanie online	wymaga dodatkowej usługi	zapewnione przez platformę	zapewnione przez platformę
Udostępnianie materiału	wymaga dodatkowego mechanizmu	zapewnione przez platformę	zapewnione przez platformę
Kontrola nad procesem tworzenia	wysoka	zależna od użytej aplikacji	zazwyczaj ograniczona

Przedstawione metodyki różnią się zatem przede wszystkim miejscem realizacji poszczególnych etapów obsługi materiału oraz zakresem odpowiedzialności aplika-

cji lokalnej i infrastruktury serwerowej. Model lokalny zapewnia największą kontrolę nad procesem tworzenia materiału, lecz samodzielnie nie dostarcza mechanizmów jego dalszej dystrybucji. Model oparty na przesyłaniu gotowego pliku rozdziela proces tworzenia od jego późniejszej obsługi, umożliwiając niezależne wykorzystanie aplikacji rejestrującej i platformy przechowującej materiał. Model zintegrowany upraszcza natomiast cały proces z perspektywy użytkownika, ponieważ poszczególne etapy są realizowane w ramach jednego ekosystemu.

Analiza przedstawionych podejść wskazuje, że możliwe jest również połączenie ich najważniejszych cech. Aplikacja lokalna może zapewniać użytkownikowi większą kontrolę nad procesem przechwytywania i przygotowania materiału, podczas gdy system serwerowy może odpowiadać za jego trwałe przechowywanie, zarządzanie oraz udostępnianie. Takie rozwiązanie pozwala oddzielić zadania wymagające bezpośredniego dostępu do zasobów urządzenia od funkcji związanych z dostępem do materiałów za pośrednictwem sieci.

2.3. Najbardziej istotne funkcjonalności

Przedstawiona charakterystyka istniejących rozwiązań pozwala na zestawienie ich najważniejszych funkcjonalności w kontekście tematu pracy. W celu zachowania spójności analizy uwzględniono przede wszystkim funkcje związane z procesem tworzenia, przetwarzania, przesyłania, przechowywania oraz udostępniania materiału wideo. Takie ujęcie pozwala porównać rozwiązania reprezentujące wyżej przedstawione modele działania.

Tabela 2.2: Zestawienie wymagań funkcjonalnych wybranych rozwiązań

Funkcjonalność	**YouTube**	**Vimeo**	**Loom**	**Microsoft Teams**	**Google Meet**	**OBS Studio**
Przechwytywanie obrazu ekranu	–	✓	✓	✓	✓	✓
Przechwytywanie obrazu z kamery	–	✓	✓	✓	✓	✓
Rejestrowanie dźwięku	–	✓	✓	✓	✓	✓
Wybór źródła obrazu	–	✓	✓	✓	✓	✓
Jednoczesne wykorzystanie wielu źródeł	–	✓	✓	✓	✓	✓

Kontynuacja na następnej stronie

Funkcjonalność	**YouTube**	**Vimeo**	**Loom**	**Microsoft Teams**	**Google Meet**	**OBS Studio**
Konfiguracja parametrów nagrywania	–	✓	✓	ograniczona	ograniczona	✓
Lokalny zapis nagrania	–	–	ograniczony	zależny od konfiguracji	ograniczony	✓
Enkodowanie materiału	–	✓	✓	✓	✓	✓
Przesyłanie gotowego pliku	✓	✓	✓	✓	✓	zależny od konfiguracji
Zdalne przechowywanie materiałów	✓	✓	✓	✓	✓	–
Biblioteka przechwyconych materiałów	✓	✓	✓	✓	✓	–
Odtwarzanie w przeglądarce	✓	✓	✓	✓	✓	–
Udostępnianie materiału za pomocą odnośnika	✓	✓	✓	✓	✓	–
Zarządzanie własnymi materiałami	✓	✓	✓	✓	✓	lokalne pliki
Uwierzytelnianie użytkownika	✓	✓	✓	✓	✓	–
Dostęp z poziomu aplikacji webowej	✓	✓	✓	✓	✓	–
Dostęp z poziomu aplikacji desktopowej	✓	już niewspierany	✓	✓	✓	✓
Wznawianie transferu plików	✓	✓	✓	✓	–	–
Obsługa materiałów o dużym rozmiarze	✓	✓	✓	✓	✓	✓

Na podstawie przedstawionego zestawienia (tabela 2.2) można zauważyć, że żadne z analizowanych rozwiązań nie koncentruje się jednocześnie na zapewnieniu rozbudowanej kontroli nad lokalnym procesem rejestrowania oraz równoczesnym za-

pewnieniu niezależnego systemu zdalnego przechowywania i zarządzania materiałami. Poszczególne platformy realizują te zadania w w różnym zakresie, dostosowanym do ich podstawowego przeznaczenia.

2.4. Wnioski z analizy

Analizowane rozwiązania można podzielić między innymi ze względu na miejsce realizacji procesu rejestrowania, sposób przechowywania materiałów oraz zakres funkcji związanych z ich dalszym udostępnianiem.

Aplikacje przeznaczone do lokalnego rejestrowania obrazu, zapewniają użytkownikowi szerokie możliwości konfiguracji procesu przechwytywania oraz przygotowania materiału. Jednocześnie tego rodzaju rozwiązania nie koncentrują się na późniejszym przechowywaniu i udostępnianiu nagrań.

Odmienne podejście prezentują platformy skoncentrowane wokół obsługi gotowych materiałów. Zapewniają one infrastrukturę przeznaczoną do przesyłania, przechowywania, przetwarzania, odtwarzania i udostępniania plików wideo. W tym modelu proces tworzenia materiału jest jednak oddzielony od platformy odpowiedzialnej za jego dalszą obsługę, co umożliwia korzystanie z różnych narzędzi rejestrujących. Wymaga to jednak dodatkowego etapu przekazania gotowego materiału do usługi.

Rozwiązania takie jak *Loom* pokazują możliwość połączenia obu opisanych modeli. Proces tworzenia materiału jest zintegrowany z jego późniejszym przechowywaniem i udostępnianiem, dzięki czemu użytkownik może przejść od rejestrowania do udostępnienia nagrania bez konieczności korzystania z niezależnych usług.

Z kolei platformy komunikacyjne integrują rejestrowanie z komunikacją w czasie rzeczywistym, traktując nagranie jako jeden z rezultatów prowadzonego spotkania.

Poszczególne rozwiązania różnią się zakresem kontroli oferowanej użytkownikowi oraz stopniem integracji kolejnych etapów obsługi materiału. Większa kontrola nad procesem lokalnym wiąże się zazwyczaj z koniecznością samodzielnego rozwiązania problemów związanych z przechowywaniem i udostępnianiem plików. Z kolei wysoki poziom integracji zapewniany przez zintegrowane ekosystemy upraszcza obsługę z perspektywy użytkownika, lecz może ograniczać kontrolę nad technicznymi aspektami procesu tworzenia materiału.

Na podstawie przeprowadzonej analizy przyjęto, że projektowany system powinien łączyć wybrane cechy obu podejść. Aplikacja desktopowa powinna zapewniać możliwość lokalnego przechwytywania obrazu i dźwięku, przygotowania wynikowego materiału wideo oraz przesyłu do części webowej systemu. Natomiast aplikacja webowa powinna odpowiadać za jego dalszą obsługę oraz kontrolę dostępu do treści. Taki podział pozwala wykorzystać możliwości środowiska desktopowego w zakresie

dostępu do urządzeń i zasobów systemowych, jednocześnie zapewniając użytkownikom wygodny dostęp do materiałów za pośrednictwem przeglądarki internetowej.

Istotnym wnioskiem z analizy jest również potrzeba rozdzielenia odpowiedzialności pomiędzy poszczególne komponenty systemu. Proces rejestrowania i przygotowania materiału wymaga bezpośredniego dostępu do zasobów urządzenia użytkownika, podczas gdy długoterminowe przechowywanie, zarządzanie oraz udostępnianie plików może być realizowane niezależnie po stronie serwera. Przyjęcie takiego podziału umożliwia niezależny rozwój obu części systemu oraz ogranicza wzajemne zależności pomiędzy mechanizmami odpowiedzialnymi za rejestrowanie i obsługę gotowych materiałów.

Przeprowadzona analiza stanowi tym samym podstawę do określenia wymagań stawianych projektowanemu systemowi. Szczególne znaczenie mają wymagania związane z rejestrowaniem obrazu i dźwięku, przygotowaniem materiału wideo, jego niezawodnym przekazaniem do serwera, przechowywaniem oraz późniejszym odtwarzaniem i udostępnianiem. Wymagania te, wraz z ograniczeniami wynikającymi z przyjętej architektury, zostaną określone w dalszej części pracy.

Rozdział 3.

Analiza problemu i wymagań systemu

Przeprowadzona analiza istniejących rozwiązań wykazała, że proces tworzenia oraz dalszej obsługi materiałów wideo może być realizowany na wiele sposobów. Dostępne systemy koncentrują się jednak zazwyczaj na wybranym etapie tego procesu. Aplikacje desktopowe, zapewniają szerokie możliwości lokalnego przechwytywania i przygotowania materiału, natomiast platformy internetowe, koncentrują się przede wszystkim na przechowywaniu, odtwarzaniu i udostępnianiu gotowych plików. Istnieją również rozwiązania integrujące oba obszary, jednak sposób realizacji poszczególnych funkcjonalności jest ściśle związany z przeznaczeniem konkretnej platformy.

3.1. Kontekst projektowanego systemu

Na podstawie przeprowadzonej analizy przyjęto, że problem projektowy rozpatrywany w niniejszej pracy polega na opracowaniu systemu umożliwiającego połączenie lokalnego procesu tworzenia materiału wideo z jego zdalnym przesyłaniem, przechowywaniem oraz późniejszą obsługą za pośrednictwem aplikacji webowej. Istotnym założeniem jest przy tym rozdzielenie odpowiedzialności pomiędzy dwa niezależne komponenty. Aplikacja desktopowa powinna odpowiadać za zadania wymagające bezpośredniego dostępu do zasobów urządzenia użytkownika, natomiast aplikacja webowa powinna zapewniać funkcje związane z zarządzaniem materiałami po ich przekazaniu do serwera.

Takie rozdzielenie wynika przede wszystkim z odmiennego charakteru obu grup funkcjonalności. Przechwytywanie obrazu i dźwięku wymaga dostępu do urządzeń oraz mechanizmów systemu operacyjnego, a także lokalnego przetwarzania danych multimedialnych. Z kolei przechowywanie i udostępnianie gotowych materiałów wymaga mechanizmów pozwalających na zarządzanie danymi, kontrolę dostępu oraz zapewnienie użytkownikom dostępu do zasobów za pośrednictwem sieci. Umiesz-

czenie wszystkich tych funkcji w jednym komponencie systemu nie jest konieczne i prowadziłoby do połączenia odpowiedzialności o odmiennym charakterze.

Projektowany system jest przeznaczony przede wszystkim dla użytkownika indywidualnego, który chce utworzyć materiał przedstawiający zawartość ekranu wraz z dźwiękiem, a następnie zachować go w centralnym miejscu i uzyskać możliwość dostępu do niego za pośrednictwem przeglądarki internetowej.
Typowy sposób wykorzystania systemu zakłada rozpoczęcie procesu rejestrowania przy użyciu aplikacji desktopowej, utworzenie lokalnego pliku wynikowego, a następnie przekazanie go do części serwerowej. Po zakończeniu przesyłania materiał staje się dostępny w aplikacji webowej, która umożliwia jego dalszą obsługę, w szczególności przeglądanie, odtwarzanie, udostępnianie szerszej grupie użytkowników oraz usuwanie.

Istotnym elementem przyjętego modelu jest rozdzielenie procesu tworzenia materiału od procesu jego przesyłania. Utworzenie pliku wideo odbywa się lokalnie, dzięki czemu operacje związane z przechwytywaniem, synchronizacją, enkodowaniem oraz multipleksingiem nie wymagają stałego połączenia z serwerem. Dopiero gotowy materiał wynikowy jest przekazywany do części webowej systemu. Pozwala to na niezależne traktowanie procesu rejestrowania i procesu transferu, co ma znaczenie w przypadku wystąpienia problemów z połączeniem sieciowym.

Ze względu na potencjalnie duży rozmiar plików multimedialnych szczególne znaczenie ma sposób realizacji ich przesyłania. W przeciwieństwie do niewielkich danych tekstowych pliki wynikowe mogą osiągać rozmiary, dla których jednorazowe przesłanie całej zawartości staje się podatne na problemy wynikające z niestabilności połączenia lub jego czasowego przerwania. Z tego względu projektowany system powinien uwzględniać możliwość kontrolowania stanu procesu przesyłania oraz jego wznowienia bez konieczności ponownego przekazywania danych, które zostały już poprawnie dostarczone do serwera.

Przyjęty sposób działania wiąże się również z koniecznością zapewnienia odpowiedniej kontroli dostępu do materiałów. Pliki przesyłane przez użytkowników nie powinny być traktowane jako zasoby publiczne dostępne bez uwierzytelnienia do momentu zatwierdzenia zmiany tego stanu przez twórcę. System powinien umożliwiać identyfikację użytkownika oraz powiązanie utworzonych przez niego materiałów z jego kontem. Pozwala to na ograniczenie dostępu do operacji związanych z zarządzaniem materiałami do uprawnionych użytkowników.

Projektowane rozwiązanie **nie ma** na celu zastąpienia rozbudowanych platform do prowadzenia wideokonferencji ani profesjonalnych aplikacji przeznaczonych do zaawansowanej produkcji materiałów wideo. Jego zadaniem jest połączenie podstawowych mechanizmów lokalnego tworzenia materiałów z mechanizmami ich transferu oraz obsługi, celem utworzenia szkieletu systemu do dalszego rozwoju. Zakres ten pozwala skoncentrować się na zagadnieniach związanych z integracją aplikacji desktopowej i webowej, przetwarzaniem danych multimedialnych, komunikacją sieciową

oraz obsługą dużych plików.

Przyjęty kontekst projektowy stanowi podstawę do określenia grupy użytkowników oraz sposobu ich interakcji z systemem. Na tej podstawie w kolejnej sekcji przedstawiono charakterystykę użytkowników projektowanego rozwiązania, a następnie zdefiniowano podstawowe scenariusze jego wykorzystania.

3.2. Analiza użytkowników systemu

Projektowany system jest przeznaczony dla grupy użytkowników indywidualnych korzystających z komputera stacjonarnego wyposażonego w dostęp do sieci oraz środowisko umożliwiające uruchomienie aplikacji desktopowej. W początkowej fazie projektu skupiono się na obsłudze systemów operacyjnych rodziny Microsoft Windows, z planem na dalsze rozszerzenie wsparcia dla macOS oraz systemów operacyjnych opartych na jądrze Linux. Użytkownik nie musi posiadać specjalistycznej wiedzy technicznej dotyczącej przetwarzania materiałów multimedialnych, ponieważ podstawowe operacje związane z przechwytywaniem, tworzeniem oraz przesyłaniem materiałów są realizowane przez aplikację. Podstawowym typem użytkownika systemu jest użytkownik posiadający konto w części webowej. Konto umożliwia identyfikację użytkownika oraz powiązanie z nim materiałów utworzonych i przesłanych w ramach systemu. Uwierzytelnienie jest wykorzystywane zarówno podczas korzystania z aplikacji webowej, jak i podczas komunikacji aplikacji desktopowej z częścią serwerową systemu.

Użytkownik korzysta z obu komponentów systemu, jednak zakres wykonywanych przez niego operacji zależy od wykorzystywanego interfejsu. Aplikacja desktopowa służy przede wszystkim do przechwytywania obrazu i dźwięku oraz przygotowania i przesyłania materiału wideo. Aplikacja webowa zapewnia natomiast dostęp do materiałów zapisanych po stronie serwera oraz umożliwia wykonywanie operacji związanych z ich dalszą obsługą.

Typowy sposób korzystania z systemu rozpoczyna się od uruchomienia aplikacji desktopowej. Użytkownik może rozpocząć proces rejestrowania, podczas którego system przechwytuje dane obrazu i dźwięku. Po zakończeniu rejestrowania powstaje plik wynikowy. Następnie może on zostać przekazany do części serwerowej po uwierzytelnieniu użytkowanika. Po zakończeniu przesyłania użytkownik może wykorzystać aplikację webową do przeglądania dostępnych materiałów oraz wykonywania operacji przewidzianych przez system.

Ze względu na przyjęty zakres pracy nie wyróżniono odrębnych ról użytkowników, takich jak administrator czy moderator. System koncentruje się na obsłudze podstawowego scenariusza użytkownika indywidualnego, obejmującego tworzenie, przesyłanie oraz zarządzanie własnymi materiałami. Ograniczenie liczby ról upraszcza model uprawnień i pozwala skoncentrować implementację na mechanizmach

związanych z głównym celem projektowanego rozwiązania.

3.2.1. Historyjki użytkowników

Historyjki użytkowników pozwalają przedstawić funkcjonalności systemu z perspektywy użytkownika końcowego oraz określić cel, jaki ma zostać osiągnięty w wyniku wykonania danej operacji. Zdefiniowane historyjki stanowią jednocześnie podstawę do określenia wymagań funkcjonalnych przedstawionych w dalszej części pracy.[19]

Ze względu na przyjęty model użytkownika oraz zakres systemu wyróżniono następujące podstawowe historyjki użytkownika:

- **Rejestracja użytkownika** – jako użytkownik chcę utworzyć konto w systemie, aby móc korzystać z funkcjonalności wymagających uwierzytelnienia oraz przechowywać własne materiały.
- **Logowanie** – jako użytkownik chcę zalogować się do systemu, aby uzyskać dostęp do własnych materiałów oraz funkcjonalności wymagających uwierzytelnienia.
- **Nagrywanie** – jako niezarejstrowany użytkownik chcę rozpocząć i zakończyć rejestrowanie obrazu oraz dźwięku, aby utworzyć materiał wideo.
- **Przesłanie materiału** – jako zalogowany użytkownik chcę przesłać utworzony plik do systemu, aby móc przechowywać go na serwerze i korzystać z niego za pośrednictwem aplikacji webowej.
- **Wznowienie przesyłania** – jako użytkownik chcę wznowić przerwane przesyłanie materiału, aby nie musieć ponownie przesyłać danych, które zostały już poprawnie przekazane na serwer.
- **Monitorowanie przesyłania** – jako użytkownik chcę znać aktualny stan przesyłania materiału, aby móc określić, czy proces nadal trwa, został zakończony lub został przerwany.
- **Przeglądanie materiałów** – jako użytkownik chcę przeglądać listę własnych materiałów, aby móc odnaleźć wcześniej przesłane nagrania.
- **Odtwarzanie materiału** – jako użytkownik chcę odtworzyć wybrany materiał za pomocą przeglądarki internetowej, aby móc zapoznać się z jego zawartością bez konieczności pobierania pliku na urządzenie.
- **Udostępnianie materiału** – jako użytkownik chcę mieć możliwość publicznego udostępnienia przesłanych materiałów oraz przesłania kierujących do nich odnośników.

- **Usunięcie materiału** – jako użytkownik chcę usunąć wybrany materiał, aby zarządzać zawartością własnej biblioteki oraz usuwać niepotrzebne pliki.

Przedstawione historyjki obejmują podstawowy cykl życia materiału w systemie – od uwierzytelnienia użytkownika, poprzez jego utworzenie i przesłanie, aż do późniejszego wykorzystania oraz usunięcia. Uwzględniają również sytuację przerwania transferu, która ma szczególne znaczenie w przypadku przesyłania materiałów o dużym rozmiarze. Konkretne mechanizmy realizacji tych funkcjonalności zostaną przedstawione w kolejnych rozdziałach pracy.

3.3. Wymagania funkcjonalne

Na podstawie zdefiniowanych historyjek użytkownika oraz przyjętego kontekstu projektowanego rozwiązania określono wymagania funkcjonalne systemu. Opisują one operacje, które system powinien udostępniać użytkownikowi, oraz zachowanie poszczególnych komponentów podczas realizacji podstawowych scenariuszy.[20] Wymagania zostały podzielone według obszarów funkcjonalnych, co pozwala powiązać je z poszczególnymi etapami korzystania z systemu.

Zarządzanie kontem i uwierzytelnianie

- **WF-01** System powinien umożliwiać użytkownikowi utworzenie konta przy użyciu adresu poczty elektronicznej oraz hasła.
- **WF-02** System powinien umożliwiać użytkownikowi uwierzytelnienie przy użyciu danych przypisanych do jego konta.
- **WF-03** System powinien umożliwiać aplikacji desktopowej uzyskanie uprawnień do komunikacji z częścią serwerową w imieniu uwierzytelnionego użytkownika.
- **WF-04** System powinien powiązywać materiały utworzone przez użytkownika z jego kontem.

Rejestrowanie materiału

- **WF-05** Aplikacja desktopowa powinna umożliwiać rozpoczęcie rejestrowania obrazu.
- **WF-06** Aplikacja desktopowa powinna umożliwiać rejestrowanie danych dźwiękowych wraz z obrazem.
- **WF-07** Aplikacja desktopowa powinna umożliwiać zakończenie procesu rejestrowania.

- **WF-08** Aplikacja desktopowa powinna utworzyć wynikowy plik wideo na podstawie zarejestrowanych danych.
- **WF-09** System powinien zapisywać podstawowe informacje dotyczące utworzonego materiału, w szczególności jego nazwę oraz parametry niezbędne do jego dalszej obsługi.

Przesyłanie materiałów

- **WF-10** System powinien umożliwiać przesłanie materiału wideo z aplikacji desktopowej do części serwerowej.
- **WF-11** System powinien umożliwiać przesyłanie materiału w sposób pozwalający na podział danych na niezależne części.
- **WF-12** System powinien umożliwiać określenie stanu procesu przesyłania materiału.
- **WF-13** System powinien umożliwiać wznowienie przesyłania materiału po jego przerwaniu.
- **WF-14** W przypadku wznowienia przesyłania system powinien umożliwiać pominięcie danych, które zostały wcześniej poprawnie przekazane na serwer.
- **WF-15** System powinien umożliwiać zakończenie procesu przesyłania i zapisanie kompletnego materiału po stronie serwera.
- **WF-16** System powinien umożliwiać usunięcie rozpoczętego lub zakończonego procesu przesyłania przez uprawnionego użytkownika.

Zarządzanie materiałami

- **WF-17** Aplikacja webowa powinna umożliwiać użytkownikowi wyświetlenie listy należących do niego materiałów.
- **WF-18** System powinien umożliwiać wyświetlenie podstawowych informacji dotyczących wybranego materiału.
- **WF-19** System powinien umożliwiać odtworzenie kompletnego materiału wideo za pośrednictwem aplikacji webowej.
- **WF-20** System powinien umożliwiać użytkownikowi usunięcie należącego do niego materiału.
- **WF-21** Usunięcie materiału powinno powodować usunięcie odpowiadających mu danych przechowywanych po stronie serwera.

Przedstawiony zbiór wymagań funkcjonalnych opisuje podstawowy zakres działania projektowanego systemu. Obejmuje on zarówno funkcje związane z obsługą kont użytkowników i rejestrowaniem materiałów, jak również mechanizmy odpowiedzialne za ich przesyłanie, wznowienie przerwanego transferu oraz późniejsze zarządzanie zapisanymi materiałami. Szczególne znaczenie ma rozdzielenie procesu tworzenia materiału od procesu jego przesyłania, dzięki czemu oba etapy mogą być realizowane niezależnie.

3.4. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne opisują właściwości, jakie powinno posiadać projektowane rozwiązanie niezależnie od realizowanych przez nie funkcji [21]. W przypadku analizowanego systemu szczególne znaczenie mają kwestie związane z bezpieczeństwem komunikacji, niezawodnością przesyłania dużych plików, wydajnością, obsługą operacji równoległych oraz możliwością dalszego rozwoju rozwiązania.

Ze względu na charakter systemu wymagania niefunkcjonalne podzielono na sześć głównych obszarów.

Bezpieczeństwo

- **WN-01** Komunikacja pomiędzy aplikacją desktopową, a częścią serwerową powinna być realizowana z wykorzystaniem bezpiecznego połączenia sieciowego.
- **WN-02** Dostęp do operacji wymagających identyfikacji użytkownika powinien być możliwy wyłącznie po jego prawidłowym uwierzytelnieniu.
- **WN-03** System powinien weryfikować uprawnienia użytkownika przed wykonaniem operacji dotyczących należących do niego materiałów.
- **WN-04** Dane uwierzytelniające oraz dane umożliwiające dostęp do zasobów użytkownika nie powinny być przekazywane w sposób jawny.

Wydajność

- **WN-05** Mechanizm przesyłania powinien umożliwiać obsługę materiałów wideo o dużym rozmiarze bez konieczności przechowywania całego materiału w pamięci operacyjnej procesu serwerowego.
- **WN-06** Operacje przesyłania danych powinny wykorzystywać dostępne zasoby sieciowe w sposób umożliwiający efektywny transfer materiałów.
- **WN-07** Przetwarzanie materiału po stronie aplikacji desktopowej nie powinno uniemożliwiać wykonywania podstawowych operacji interfejsu użytkownika.

Niezawodność przesyłu

- **WN-08** Przerwanie połączenia sieciowego podczas przesyłania materiału nie powinno powodować utraty danych, które zostały wcześniej poprawnie zapisane po stronie serwera.
- **WN-09** System powinien umożliwiać określenie stanu nieukończonego procesu przesyłania po wystąpieniu błędu lub przerwaniu połączenia.
- **WN-10** Wznowienie przesyłania powinno umożliwiać kontynuowanie transferu od miejsca, w którym został on przerwany.
- **WN-11** System powinien zapewniać spójność kompletnego pliku wynikowego po zakończeniu przesyłania wszystkich jego części.

Obsługa operacji równoległych

- **WN-12** System powinien umożliwiać jednoczesną obsługę wielu procesów przesyłania.
- **WN-13** Równoległe przesyłanie danych nie powinno powodować przypisania części jednego materiału do innego procesu przesyłania.
- **WN-14** System powinien zachowywać spójność informacji o stanie poszczególnych procesów przesyłania w przypadku wykonywania operacji równolegle.

Dostępność i użyteczność

- **WN-15** Funkcjonalności związane z zarządzaniem materiałami powinny być dostępne za pośrednictwem standardowej przeglądarki internetowej.
- **WN-16** Interfejs aplikacji webowej powinien umożliwiać użytkownikowi jednoznaczne rozpoznanie dostępnych operacji związanych z materiałami.
- **WN-17** Aplikacja desktopowa powinna informować użytkownika o podstawowym stanie procesu rejestrowania oraz przesyłania materiału.

Możliwość rozwoju i skalowalność

- **WN-18** Architektura systemu powinna umożliwiać niezależny rozwój aplikacji desktopowej oraz aplikacji webowej.
- **WN-19** Komunikacja pomiędzy komponentami powinna być oparta na jasno określonym interfejsie, umożliwiającym dodawanie kolejnych klientów korzystających z części serwerowej.

- **WN-20** Mechanizm obsługi przesyłu powinien umożliwiać rozszerzenie systemu o kolejne metody transferu plików bez konieczności przebudowy pozostałych funkcjonalności.

- **WN-21** Sposób przechowywania danych powinien umożliwiać zwiększenie liczby obsługiwanych materiałów bez zmiany podstawowego modelu funkcjonalnego systemu.

Przedstawione wymagania niefunkcjonalne określają właściwości projektowanego systemu, które mają istotny wpływ na sposób jego implementacji oraz możliwość późniejszej weryfikacji. Szczególne znaczenie przypisano niezawodności procesu przesyłu, ponieważ materiały wideo mogą charakteryzować się znacznym rozmiarem, a tym samym długim czasem transferu. Zastosowanie mechanizmu umożliwiającego wznowienie przesyłania pozwala ograniczyć skutki chwilowej utraty połączenia i uniknąć konieczności ponownego przesyłania danych.

Istotnym aspektem jest również możliwość wykonywania operacji równolegle. Projektowany system powinien być przygotowany nie tylko na obsługę pojedynczego transferu, lecz również na sytuację, w której wiele procesów przesyłu jest realizowanych jednocześnie. Wymaga to zachowania niezależności poszczególnych procesów oraz spójności ich stanu.

Wymagania dotyczące bezpieczeństwa, niezawodności, wydajności i współbieżności stanowią również podstawę do przeprowadzenia testów projektowanego rozwiązania.

3.5. Ograniczenia projektowe

Projektowanie i implementacja systemu zostały przeprowadzone w ramach określonego zakresu funkcjonalnego oraz przy założeniu wykorzystania dostępnych zasobów sprzętowych i programowych. Przyjęte ograniczenia [22] wynikają zarówno z charakteru projektowanego rozwiązania, jak i z zakresu niniejszej pracy. Ich określenie pozwala jednoznacznie wskazać funkcjonalności objęte projektem oraz obszary, które nie są przedmiotem realizacji.

Pierwszym z ograniczeń jest podział systemu na dwa niezależne komponenty: aplikację desktopową oraz aplikację webową. Aplikacja desktopowa jest odpowiedzialna za operacje wymagające bezpośredniego dostępu do zasobów urządzenia użytkownika, natomiast aplikacja webowa realizuje funkcje związane z uwierzytelnianiem, przechowywaniem oraz zarządzaniem materiałami. Przyjęcie takiej architektury oznacza konieczność zapewnienia komunikacji pomiędzy komponentami za pośrednictwem sieci oraz wprowadza zależność działania części desktopowej od dostępności części serwerowej w przypadku operacji wymagających przesłania danych.

Kolejne ograniczenie stanowi środowisko uruchomieniowe aplikacji desktopowej. Ze względu na ograniczenia czasowe aplikacja jest projektowana z uwzględnieniem bibliotek cross-platformowych, aczkolwiek ukierunkowana na konkretne środowisko operacyjne. Ograniczenie to dotyczy przede wszystkim mechanizmów odpowiedzialnych za przechwytywanie obrazu i dźwięku, które zależą od dostępnych interfejsów systemowych oraz sposobu ich obsługi przez wykorzystywane biblioteki.

Istotnym ograniczeniem jest również sposób przechowywania materiałów po stronie serwerowej. W ramach pracy zastosowano lokalny system plików jako miejsce przechowywania danych binarnych materiałów wideo, natomiast informacje opisujące materiały oraz ich stan są przechowywane w bazie danych. Takie rozwiązanie upraszcza implementację oraz pozwala na realizację i testowanie mechanizmów systemu bez konieczności wykorzystania zewnętrznej usługi przechowywania obiektów *BLOB* [14] takiej jak Azure Blob Storage [23]. Jednocześnie oznacza ono zależność rozwiązania od dostępnej przestrzeni dyskowej oraz infrastruktury serwera, a także ogranicza możliwość bezpośredniego wykorzystania rozproszonych mechanizmów przechowywania danych bez modyfikacji wynikowej implementacji.

Kolejne ograniczenie wynika z zakresu obsługiwanych materiałów. Projekt koncentruje się na materiałach wideo generowanych przez aplikację desktopową oraz na mechanizmie ich przesyłania i późniejszej obsługi. Nie obejmuje natomiast pełnego procesu profesjonalnej postprodukcji, zaawansowanej edycji obrazu ani rozbudowanych mechanizmów konwersji materiałów pomiędzy wieloma formatami. Zadaniem systemu jest przede wszystkim utworzenie materiału, jego przekazanie do serwera oraz zapewnienie możliwości dalszego zarządzania nim.

Projektowany system aktualnie nie jest przeznaczony do realizacji komunikacji audio-wideo w czasie rzeczywistym. Funkcjonalność rejestrowania obrazu i dźwięku jest realizowana jako proces lokalny, którego wynikiem jest plik multimedialny. System nie obejmuje zatem mechanizmów charakterystycznych dla platform wideokonferencyjnych, takich jak transmisja obrazu pomiędzy uczestnikami spotkania, obsługa wielu strumieni w czasie rzeczywistym czy synchronizacja komunikacji pomiędzy wieloma użytkownikami.

Ostatnim istotnym ograniczeniem jest zakres niniejszej pracy. Celem projektu jest opracowanie i implementacja spójnego rozwiązania łączącego aplikację desktopową oraz webową, a nie stworzenie kompletnej, komercyjnej platformy do obsługi materiałów multimedialnych. Z tego względu niektóre zagadnienia, takie jak rozbudowane mechanizmy administracyjne, zaawansowane zarządzanie uprawnieniami, skalowanie infrastruktury na wiele serwerów czy wykorzystanie zewnętrznych usług przechowywania danych, pozostają poza zakresem realizacji.

Przyjęte ograniczenia wyznaczają zatem granice projektowanego rozwiązania i pozwalają właściwie interpretować wyniki jego implementacji oraz testów. Jednocześnie wskazują obszary, które mogą stanowić podstawę dalszego rozwoju systemu.

Rozdział 4.

Projekt systemu

4.1. Architektura systemu

4.1.1. Zarys ogólny architektury

System został zaprojektowany jako rozwiązanie składające się z dwóch głównych części: aplikacji desktopowej oraz aplikacji webowej. Podział ten wynika przede wszystkim z odmiennego charakteru operacji realizowanych przez obie części systemu. Aplikacja desktopowa wykonuje operacje wymagające bezpośredniego dostępu do zasobów sprzętowych użytkownika, natomiast aplikacja webowa stanowi centralny element systemu odpowiedzialny za zarządzanie użytkownikami, materiałami oraz komunikację z aplikacją desktopową.

Aplikacja desktopowa odpowiada za lokalne przechwytywanie obrazu i dźwięku, ich przetwarzanie i enkodowanie oraz utworzenie wynikowego pliku multimedialnego. Wykonywanie tych operacji po stronie klienta pozwala wykorzystać zasoby sprzętowe użytkownika oraz wykluczyć konieczność przesyłania nieprzetworzonych danych multimedialnych do serwera.

Aplikacja webowa odpowiada za funkcje wymagające centralnego przechowywania i udostępniania danych. Udostępnia interfejs użytkownika zrealizowany z wykorzystaniem Razor Pages oraz interfejs REST API przeznaczony do komunikacji z aplikacją desktopową. Odpowiada również za uwierzytelnianie i autoryzację użytkowników, zarządzanie materiałami oraz obsługę procesów przesyłania plików.

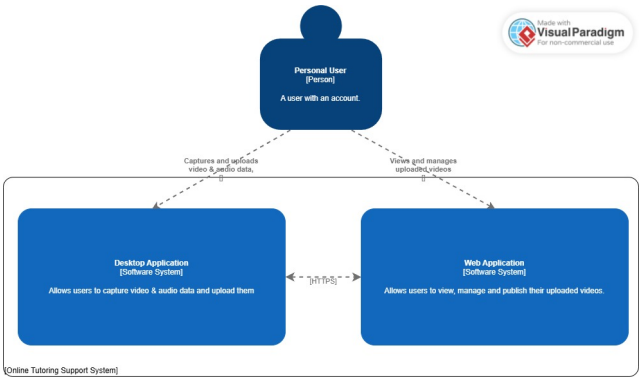
Komunikacja pomiędzy aplikacjami odbywa się za pośrednictwem protokołu HTTPS. Dostęp aplikacji desktopowej do operacji wymagajacych autoryzacji jest zabezpieczony za pomocą tokenu dostępu uzyskanego w procesie uwierzytelniania opartym na protokole OAuth 2.0 Authorization Code Flow wraz z mechanizmem Proof Key for Code Exchange.

W zakresie przechowywania danych zastosowano rozdzielenie metadanych od

ich zawartości. Dane użytkowników, operacji przesyłania oraz metadane plików przechowywane są w relacyjnej bazie danych, natomiast właściwe pliki multimedialne są przechowywane w systemie plików serwera. Przyjęte założenia pozwalają rozdzielić odpowiedzialności pomiędzy poszczególne elementy systemu, ograniczyć zależności pomiędzy aplikacją desktopową i webową oraz dostosować sposób przetwarzania danych do charakteru wykonywanych operacji.

4.1.2. Diagram kontekstu systemu – C4 System Context

Diagram kontekstu systemu (rys. 4.1) przedstawia projektowane rozwiązanie na najwyższym poziomie abstrakcji. Jego celem jest określenie granic systemu oraz przedstawienie podstawowych zależności pomiędzy systemem, a podmiotami znajdującymi się w jego otoczeniu. Zgodnie z założeniami modelu C4 szczegóły dotyczące wewnętrznej budowy systemu zostały na tym poziomie pominięte. [24]



Rysunek 4.1: Diagram kontekstu systemu projektowanego rozwiązania

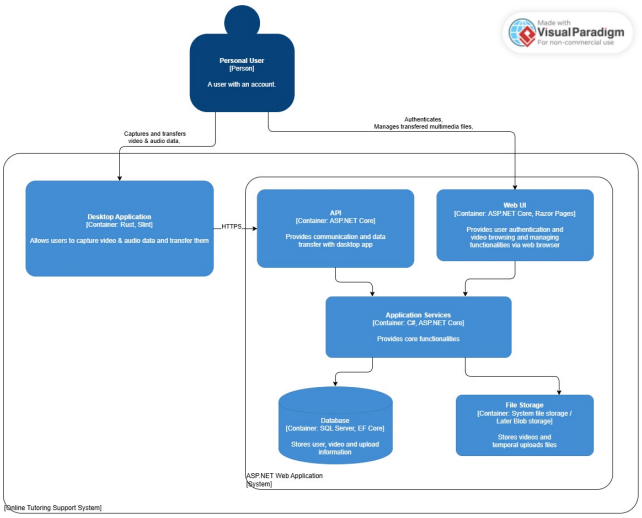
Głównym użytkownikiem systemu jest aktor korzystający z jego funkcji za pośrednictwem aplikacji desktopowej oraz webowej. Aplikacja desktopowa jest przeznaczona przede wszystkim do rejestrowania materiałów multimedialnych oraz ich transferu. Aplikacja webowa zapewnia natomiast dostęp do materiałów multimedialnych, metod zarządzania nimi oraz obsługę procesów realizowanych po stronie serwera.

Na tym poziomie obie aplikacje stanowią integralną część systemu i reprezentują jedynie sposoby interakcji użytkownika z nim. Szczegółową organizację poszczególnych części systemu przedstawiono na diagramach kontenerów i komponentów.

4.1.3. Diagram kontenerów – C4 Container

Diagram kontenerów (rys. 4.2) przedstawia wewnętrzną strukturę projektowanego systemu oraz główne elementy odpowiedzialne za realizację jego funkcjonalności. W odróżnieniu od diagramu kontekstu, na tym poziomie przedstawiono podsta-

wowe aplikacje oraz mechanizmy przechowywania danych, a także zależności pomiędzy nimi. [25]



Rysunek 4.2: Diagram kontenerów systemu projektowanego rozwiązania

Aplikacja desktopowa stanowi klienta systemu przeznaczonego przede wszystkim do przechwytywania obrazu i dźwięku, przygotowania materiału multimedialnego oraz jego przesłania. Aplikacja webowa pełni centralną rolę w systemie, udostępniając zarówno interfejs użytkownika realizowany za pomocą Razor Pages, jak i REST API wykorzystywany przez aplikację desktopową.

Logika związana z obsługą materiałów oraz procesem transferu została wydzielona w ramach usług aplikacyjnych. Dane opisujące użytkowników, multimedia oraz operacje przesyłania są przechowywane w relacyjnej bazie danych. Dostęp do bazy realizowany jest za pomocą Entity Framework Core.

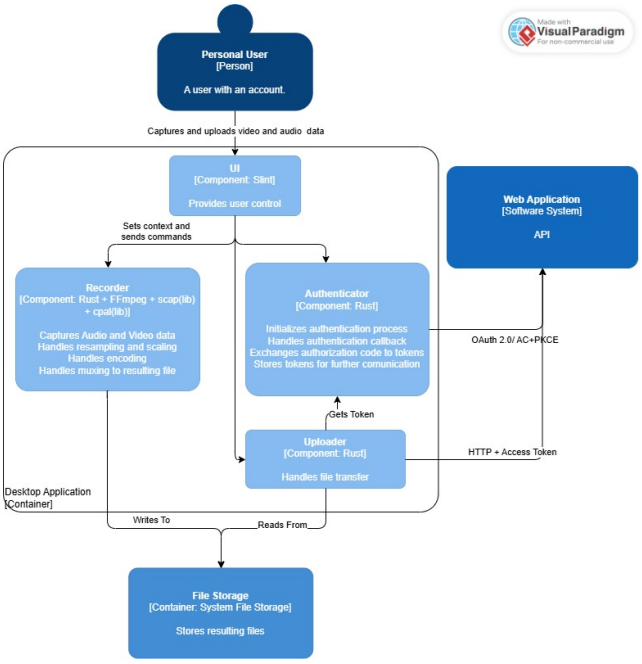
Właściwa zawartość plików multimedialnych jest przechowywana niezależnie od danych relacyjnych w serwerowym magazynie plików. Aplikacja webowa odpowiada za powiązanie informacji zapisanych w bazie z odpowiadającymi im plikami.

Komunikacja pomiędzy aplikacją desktopową a aplikacją webową odbywa się za pośrednictwem protokołu HTTPS. REST API wykorzystuje tokeny dostępu uzyskiwane w procesie uwierzytelniania opartym na OAuth 2.0 i OpenID Connect.

4.1.4. Diagram komponentów – C4 Component

Diagram komponentów stanowi trzeci poziom szczegółowości modelu C4 i przedstawia wewnętrzną strukturę wybranego kontenera zaprezentowanego na diagramie kontenerów. Jego celem jest zobrazowanie głównych komponentów tworzących aplikację, zakresu odpowiedzialności poszczególnych komponentów oraz zależności występujących pomiędzy nimi. [26]

Aplikacja Desktopowa



Rysunek 4.3: Diagram komponentów aplikacji desktopowej

Na poziomie komponentów aplikacji desktopowej wyróżniono trzy główne odpowiedzialności: uwierzytelnianie użytkownika, rejestrowanie materiału oraz jego transfer(rys. 4.3) .

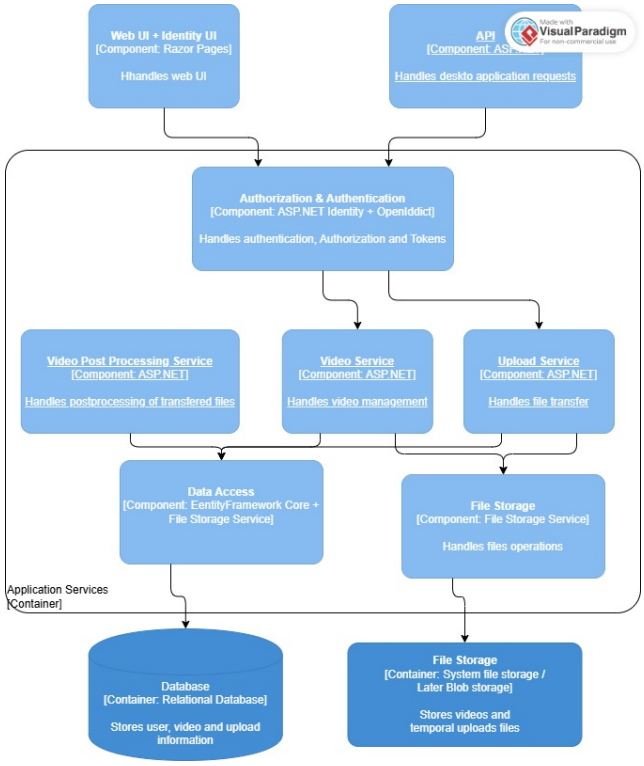
Komponent Recorder odpowiada za cały proces przygotowania pliku multimedialnego, począwszy od przechwycenia danych obrazu i dźwięku, poprzez ich przetwarzanie, aż do utworzenia pliku wynikowego.

Komponent Uploader odpowiada za komunikację z REST API oraz realizację procesu przesyłania materiału. W zależności od zastosowanego sposobu przesyłania może przekazywać plik jako całościowy lub fragmentaryczny byt. Stan operacji przesyłania jest uzyskiwany z aplikacji webowej.

Komponent Authenticator realizuje proces uwierzytelniania użytkownika z wykorzystaniem OAuth 2.0 Authorization Code Flow oraz PKCE. Po uzyskaniu tokenu dostępu jest on wykorzystywany przez komponent Uploader podczas komunikacji z endpointami API wymagającymi autoryzacji.

Aplikacja webowa

W aplikacji webowej wydzielono komponenty odpowiadające za prezentację danych, komunikację z klientem desktopowym, logikę biznesową, uwierzytelnianie oraz dostęp do zasobów. (rys. 4.4) Komponent Razor Pages odpowiada za obsługę webowego interfejsu użytkownika, natomiast REST API stanowi punkt komunikacyjny



Rysunek 4.4: Diagram komponentów aplikacji webowej

dla aplikacji desktopowej.

Upload Service realizuje logikę związaną z procesem transsferu plików. Odpowiada między innymi za utworzenie operacji przesyłania, obsługę fragmentów pliku, aktualizowanie stanu operacji oraz finalizację transferu. Video Service odpowiada natomiast za operacje związane z materiałami wideo, w szczególności ich tworzenie, pobieranie i zarządzanie metadanymi.

Komponent Data Access zapewnia dostęp do relacyjnej bazy danych za pośrednictwem Entity Framework Core. File Storage odpowiada za fizyczne przechowywanie plików multimedialnych. Rozdzielenie tych dwóch mechanizmów pozwala oddzielić metadane materiałów od ich dużej zawartości binarnej.

Authentication & Authorization odpowiada za uwierzytelnianie użytkowników oraz kontrolę dostępu do chronionych zasobów. Wykorzystuje ASP.NET Core Identity do zarządzania użytkownikami oraz OpenIddict do realizacji mechanizmu OAuth 2.0/OpenID Connect wykorzystywanego przez aplikację desktopową.

4.2. Model Danych

Model danych określa sposób reprezentacji informacji wykorzystywanych przez system oraz zależności pomiędzy poszczególnymi elementami tych informacji. [27]

W projektowanym systemie model danych został opracowany przede wszystkim na potrzeby aplikacji webowej, która odpowiada za zarządzanie użytkownikami, materiałami wideo oraz procesami ich przesyłania. Model został odwzorowany w relacyjnej bazie danych Microsoft SQL Server z wykorzystaniem Entity Framework Core.

Z punktu widzenia funkcjonalności systemu najważniejsze elementy modelu stanowią użytkownik, materiał wideo oraz proces jego przesyłania. Dodatkowo, w przypadku wykorzystania fragmentarycznego sposobu transferu pliku, proces przesyłania powiązany jest z informacjami dotyczącymi poszczególnych fragmentów. Rozdzielenie materiału wideo od procesu jego transferu pozwala niezależnie przechowywać informacje opisujące materiał oraz informacje związane z przebiegiem jego przesyłania.

Model danych obejmuje również dane związane z uwierzytelnianiem i autoryzacją użytkowników oraz obsługą protokołu OAuth 2.0/OpenID Connect. Elementy te są jednak realizowane przez wykorzystane mechanizmy ASP.NET Core Identity i OpenIddict, dlatego nie stanowią przedmiotu przedstawionego modelu. Na diagramie skoncentrowano się na encjach bezpośrednio związanych z funkcjonalnością systemu oraz ich zależnościach.

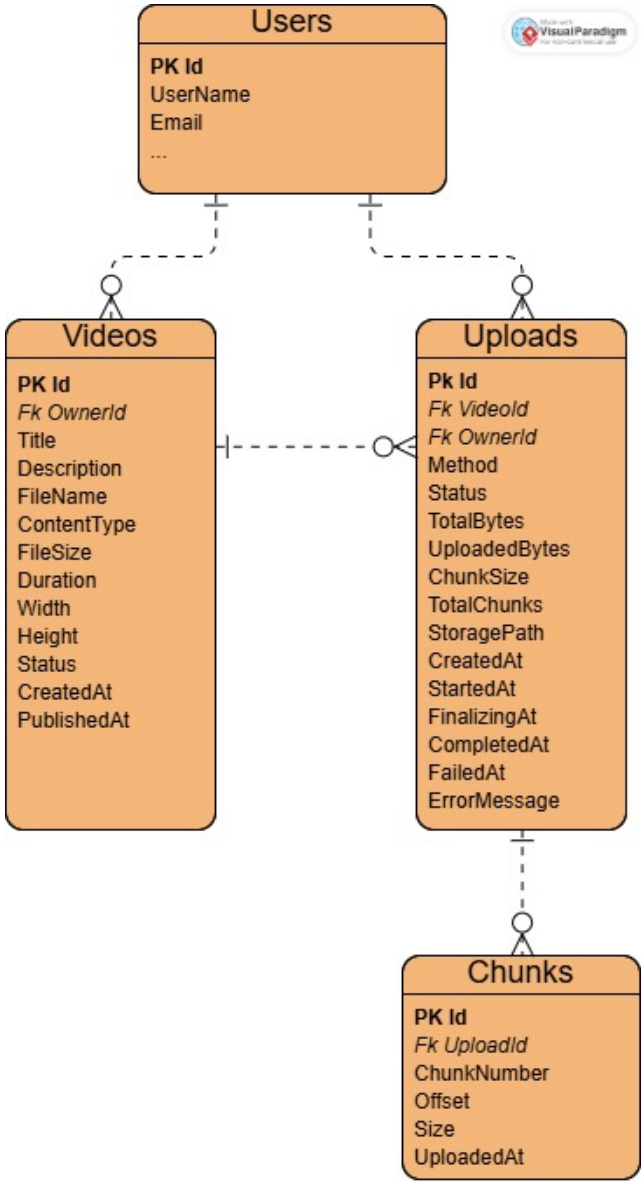
Relacje pomiędzy najważniejszymi encjami modelu przedstawiono na diagramie ERD (rys. 4.5).

4.2.1. Struktura modelu danych

Podstawową encją modelu jest Videos, reprezentująca plik multimedialny znajdujący się w systemie. Encja przechowuje informacje niezbędne do identyfikacji i prezentacji materiału – między innymi jego tytuł, opis, id oraz nazwę pliku. Zawiera również podstawowe informacje dotyczące metadanych materiału, takie jak typ zawartości, rozmiar pliku, czas trwania oraz rozdzielczość. Atrybuty CreatedAt i PublishedAt pozwalają określić odpowiednio moment utworzenia oraz opublikowania materiału, natomiast Status określa jego aktualny stan przetwarzania.

Każdy plik multimedialny jest przypisany do użytkownika za pomocą klucza obcego OwnerId. Relacja pomiędzy Users i Video ma charakter jeden-do-wielu, ponieważ pojedynczy użytkownik może posiadać wiele materiałów, podczas gdy konkretny materiał posiada jednego właściciela.

Proces przesyłania materiału jest reprezentowany przez encję Uploads. Został on celowo oddzielony od encji Videos, ponieważ przesyłanie stanowi niezależny proces, którego stan może zmieniać się w czasie. Encja Uploads przechowuje między innymi zastosowaną metodę przesyłania, aktualny stan procesu, całkowity rozmiar materiału oraz ilość przesłanych danych. Dodatkowo przechowywane są informacje dotyczące czasu rozpoczęcia, finalizacji, zakończenia lub niepowodzenia operacji oraz



Rysunek 4.5: Diagram Entity Relationship transferu plików

ewentualny komunikat błędu.

Każdy obiekt Uploads jest powiązany z konkretnym użytkownikiem poprzez OwnerId oraz z materiałem Video poprzez VideoId. Oznacza to, że użytkownik może posiadać wiele procesów przesyłania, a pojedynczy materiał może być powiązany z wieloma procesami transferu. Takie rozwiązanie pozwala rozdzielić tożsamość materiału od poszczególnych operacji związanych z jego przesyłaniem.

W przypadku fragmentarycznego przesyłania pliku wykorzystywana jest encja Chunks. Reprezentuje ona pojedynczy fragment materiału przesyłanego w ramach określonego procesu Upload. Oprócz identyfikatora procesu przechowywany jest numer fragmentu, jego przesunięcie względem początku pliku, rozmiar oraz czas przesłania. Relacja pomiędzy Uploads i Chunks ma charakter jeden-do-wielu, ponieważ

jeden proces przesyłania może składać się z wielu fragmentów.

W modelu zastosowano również ograniczenie unikalności dla pary UploadId i ChunkNumber. Zapewnia ono, że w ramach jednego procesu przesyłania nie mogą zostać zarejestrowane dwa fragmenty o tym samym numerze. Ograniczenie to wspiera spójność danych podczas równoczesnego lub ponownego przesyłania fragmentów.

Encja User jest oparta na klasie IdentityUser udostępnianej przez ASP.NET Core Identity. W związku z tym użytkownik jest jednocześnie elementem modelu odpowiedzialnym za przypisanie materiałów i procesów transferu do ich właściciela oraz częścią infrastruktury uwierzytelniania aplikacji. Pozostałe encje i tabele tworzone przez ASP.NET Core Identity oraz OpenIddict nie zostały przedstawione na diagramie, ponieważ nie są objęte zakresem tej pracy.

Model danych określa strukturę informacji, natomiast sposób ich przepływu pomiędzy aplikacją desktopową, API, bazą danych i magazynem plików przedstawiono w części dotyczącej transferu danych 4.3.

4.3. Proces Transferu Danych

Projekt transferu danych określa sposób przekazywania informacji oraz danych binarnych pomiędzy aplikacją desktopową a serwerem, a także sposób ich dalszego zapisu po stronie serwera. Ze względu na charakter systemu szczególne znaczenie ma przesyłanie plików multimedialnych, których rozmiar może być znaczny. Z tego względu proces transferu został oddzielony od operacji związanych z zarządzaniem metadanymi materiału.

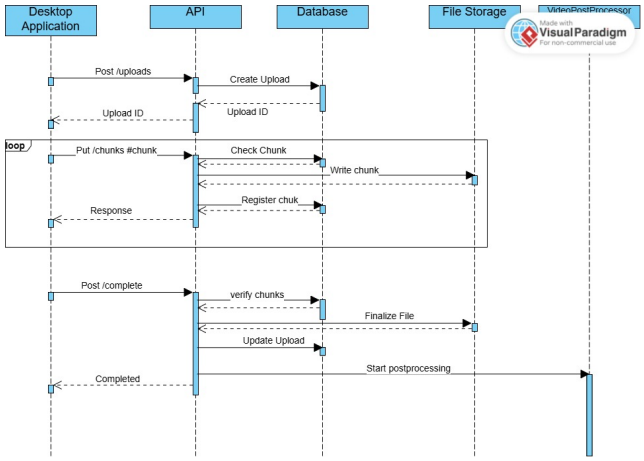
Aplikacja desktopowa pełni rolę klienta odpowiedzialnego za przygotowanie materiału oraz przekazanie go do aplikacji webowej. Komunikacja odbywa się za pośrednictwem interfejsu REST API z wykorzystaniem protokołu HTTPS. API pełni funkcję pośrednika pomiędzy klientem, relacyjną bazą danych i magazynem plików.

W procesie przesyłania rozróżniono dane opisujące operację oraz właściwe dane binarne. Informacje takie jak identyfikator operacji, jej stan, rozmiar materiału, zastosowana metoda przesyłania oraz informacje o przesłanych fragmentach są zapisywane w relacyjnej bazie danych. Zawartość pliku jest natomiast zapisywana w serwerowym magazynie plików. Pozwala to uniknąć przechowywania dużych danych binarnych bezpośrednio w bazie danych.

Podstawowym elementem organizującym proces transferu jest encja Uploads. Utworzenie obiektu tej encji rozpoczyna proces przesyłania i powoduje nadanie mu unikalnego identyfikatora. Identyfikator ten jest następnie wykorzystywany przez aplikację desktopową podczas kolejnych operacji przesyłania danych. Stan obiektu Uploads pozwala śledzić przebieg operacji niezależnie od pojedynczych żądań HTTP.

Zaimplementowano trzy metody transferu danych Multipart form data, HTTP

Streaming oraz transfer fragmentowy. Wszystkie trzy metody wykorzystują ten sam mechanizm komunikacji pomiędzy aplikacją desktopową i webową, różniąc się sposobem przekazywania danych binarnych. Encja Uploads (rys. 4.5) pozwala ujednolicić obsługę poszczególnych metod na poziomie aplikacji serwerowej.



Rysunek 4.6: Diagram sekwencji/przepływu dla przesyłu fragmentarycznego pliku

4.3.1. Transfer fragmentowy

W przypadku dużych materiałów zastosowano możliwość fragmentarycznego przesyłania pliku.

Przed rozpoczęciem właściwego transferu aplikacja desktopowa wysyła do API żądanie utworzenia operacji przesyłania. W odpowiedzi otrzymuje identyfikator encji Upload, który jednoznacznie identyfikuje daną operację.

Następnie plik jest dzielony po stronie aplikacji desktopowej na fragmenty o określonym rozmiarze. Każdy z wysyłanych współbieżnie fragmentów jest przesyłany niezależnie za pomocą żądania HTTP skierowanego do odpowiedniego zasobu API. Żądanie zawiera identyfikator operacji oraz numer przesyłanego fragmentu. Po otrzymaniu danych aplikacja webowa zapisuje je w magazynie plików oraz rejestruje informacje o przesłanym fragmencie w bazie danych.

Encja Chunks (rys. 4.5) przechowuje między innymi numer fragmentu, jego rozmiar, przesunięcie względem początku pliku oraz czas przesłania. Dzięki temu stan procesu może być określany na podstawie informacji o poszczególnych fragmentach, a przesyłanie może być realizowane niezależnie dla kolejnych części pliku.

Po przesłaniu wszystkich wymaganych fragmentów następuje etap finalizacji operacji. Aplikacja webowa weryfikuje kompletność przesłanych danych, kończy proces przesyłania oraz przygotowuje materiał do dalszego wykorzystania przez system. Informacje o zakończeniu operacji są zapisywane w encji Uploads (rys. 4.5), natomiast materiał reprezentowany przez encję Videos (rys. 4.5) może zostać udostęp-

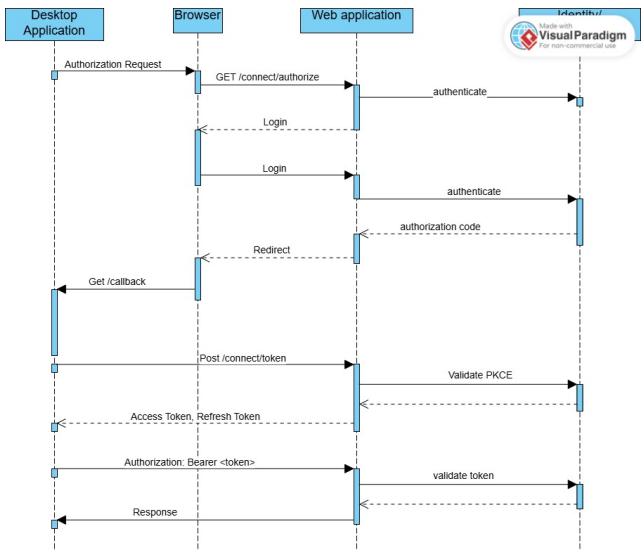
niony użytkownikowi za pośrednictwem aplikacji webowej.

4.4. Proces uwierzytelniania użytkownika

Uwierzytelnianie użytkowników w projektowanym systemie realizowane jest przez aplikację webową, która pełni rolę serwera autoryzacyjnego dla aplikacji desktopowej. Do realizacji tego procesu wykorzystano protokół OAuth 2.0 z przepływem Authorization Code oraz mechanizmem Proof Key for Code Exchange (PKCE). Implementację mechanizmu po stronie serwera zapewnia biblioteka OpenIddict, natomiast zarządzanie kontami użytkowników realizowane jest z wykorzystaniem ASP.NET Core Identity.

Przyjęte rozwiązanie pozwala oddzielić proces uwierzytelniania użytkownika od aplikacji desktopowej. Aplikacja desktopowa nie otrzymuje ani nie przechowuje hasła użytkownika. Zamiast tego inicjuje proces logowania w przeglądarce, natomiast właściwe uwierzytelnienie odbywa się w aplikacji webowej. Po jego zakończeniu aplikacja desktopowa otrzymuje autoryzacyjny kod, który następnie wymienia na token dostępu oraz token odświeżania.

Uzyskany token dostępu jest wykorzystywany podczas komunikacji aplikacji desktopowej z chronionymi zasobami REST API. Po stronie serwera token jest walidowany przez mechanizm OpenIddict, a następnie na jego podstawie ustalana jest tożsamość użytkownika wykonującego żądanie. Dzięki temu dalsza komunikacja pomiędzy aplikacją desktopową a API może odbywać się bez ponownego przekazywania danych uwierzytelniających użytkownika.



Rysunek 4.7: Diagram sekwencji uwierzytelniania użytkownika z poziomu aplikacji desktopowej

4.4.1. Przebieg procesu uwierzytelniania

Proces rozpoczyna się w aplikacji desktopowej, która generuje wymagane przez mechanizm PKCE: losowy parametr *code_verifier* oraz odpowiadający mu *code_challenge*. Następnie aplikacja otwiera domyślną przeglądarkę użytkownika i przekierowuje ją do endpointu autoryzacyjnego aplikacji webowej.

Aplikacja webowa sprawdza, czy użytkownik posiada aktywną sesję uwierzytelnioną za pomocą ASP.NET Core Identity. Jeżeli użytkownik nie jest uwierzytelniony, prezentowany jest formularz logowania. Po poprawnym podaniu danych aplikacja webowa uwierzytelnia użytkownika i przekazuje obsługę żądania do mechanizmu OpenIddict.

Po pomyślnym zakończeniu procesu autoryzacji OpenIddict generuje krótkotrwały kod autoryzacyjny. Kod ten jest przekazywany do zarejestrowanego adresu zwrotnego aplikacji desktopowej. Aplikacja desktopowa nie wykorzystuje jednak kodu bezpośrednio do wykonywania żądań API, lecz wymienia go na token dostępu oraz token odświeżania. Podczas wymiany kodu dostępu aplikacja desktopowa przekazuje również wcześniej wygenerowany *code_verifier*. OpenIddict porównuje go z wartością *code_challenge* zapisaną podczas rozpoczęcia procesu. Dopiero pozytywna weryfikacja pozwala na wydanie tokenów.

Token dostępu jest następnie dołączany przez aplikację desktopową do żądań kierowanych do chronionych zasobów API w nagłówku `Authorization`. Po stronie aplikacji webowej token jest sprawdzany przez mechanizm OpenIddict. Na jego podstawie ustalana jest tożsamość użytkownika, a następnie sprawdzane są wymagane uprawnienia do danego zasobu. Po wygaśnięciu tokenu dostępu aplikacja desktopowa może wykorzystać token odświeżania do uzyskania nowego tokenu dostępu bez konieczności ponownego przeprowadzania interaktywnego procesu logowania

Rozdział 5.

Implementacja systemu

Niniejszy rozdział przedstawia zastosowane technologie, sposób realizacji wybranych elementów zaprojektowanego systemu oraz najważniejsze problemy napotkane podczas jego implementacji.

Szczególną uwagę poświęcono części aplikacji desktopowej, w której realizowane są operacje związane z przechwytywaniem, przetwarzaniem i enkodowaniem strumieni obrazu i dźwięku. Ze względu na charakter tych operacji implementacja ta wymagała rozwiązania problemów związanych między innymi z przetwarzaniem danych w czasie rzeczywistym, współbieżnością, buforowaniem oraz synchronizacją.

Implementacja aplikacji webowej została wykonana przede wszystkim w minimalnym zakresie wymaganym do integracji z aplikacją desktopową oraz prezentacji rezultatów jej działania. W obecnej wersji systemu serwer realizuje podstawowy cykl obsługi nagrania: przyjmuje przesłany plik wideo, zapisuje go w pamięci trwałej, wykonuje uproszczoną symulację procesu przetwarzania, a następnie udostępnia nagranie użytkownikowi. Pełne przetwarzanie danych plików po stronie serwera – obejmujące między innymi analizę i ekstrakcję metadanych, weryfikację poprawności pliku oraz właściwe przetwarzanie z wykorzystaniem FFmpeg, nie zostało objęte implementacją tej wersji systemu.

Jednocześnie struktura aplikacji webowej została przygotowana z wykorzystaniem interfejsów i wydzielonych komponentów odpowiedzialnych za poszczególne operacje, dzięki czemu uproszczone mechanizmy mogą zostać zastąpione właściwą implementacją bez konieczności zmiany pozostałych elementów systemu. Pozwala to traktować obecną implementację jako funkcjonalną podstawę dla dalszego rozwoju warstwy przetwarzania po stronie serwera.

Podstawowe mechanizmy komunikacji pomiędzy aplikacją desktopową i webową, w tym REST API, obsługa uwierzytelniania i autoryzacji oraz zapis danych z wykorzystaniem Entity Framework Core, opierają się na standardowych mechanizmach dostępnych dla wykorzystanych technologii. Interesującym z punktu widzenia implementacji elementem jest natomiast mechanizm transferu plików, w którym

zastosowano fragmentację danych, śledzenie stanu przesyłania oraz obsługę równoległych żądań.

Najbardziej złożonym elementem systemu jest aplikacja desktopowa oraz zaimplementowany w niej potok przetwarzania multimediów. Wykorzystując biblioteki odpowiedzialne za przechwytywanie obrazu i dźwięku oraz framework FFmpeg, zaimplementowano współbieżne przetwarzanie niezależnych strumieni danych, ich enkodowanie oraz końcowe multipleksowanie do pliku wynikowego. W tej części szczególne znaczenie mają ograniczenia wydajnościowe związane z programowym przetwarzaniem obrazu i dźwięku. Ich konsekwencją może być między innymi wzrost opóźnienia, utrata klatek oraz zwiększone zużycie pamięci. Zagadnienia te, wraz z możliwościami wykorzystania akceleracji sprzętowej, stanowią główny przedmiot dalszej części rozdziału.

5.1. Wybrane narzędzia oraz technologie

System został zaimplementowany jako rozwiązanie składające się z aplikacji webowej oraz aplikacji desktopowej. Ze względu na różny charakter zadań realizowanych przez obie części systemu zastosowano odmienne technologie i biblioteki.

5.1.1. Aplikacja webowa

- C — język programowania wykorzystany do implementacji logiki aplikacji webowej. [30]
- .NET 8 — platforma programistyczna stanowiąca podstawę aplikacji webowej. [31]
- ASP.NET Core — framework wykorzystany do implementacji aplikacji webowej, obsługi żądań HTTP oraz interfejsu API.[32]
- Razor Pages — mechanizm ASP.NET Core wykorzystany do implementacji stron interfejsu użytkownika aplikacji. [33]
- Entity Framework Core — obiektowo-relacyjny mapper (ORM) wykorzystany do komunikacji z bazą danych oraz zarządzania encjami i migracjami. [34]
- Microsoft SQL Server — relacyjny system zarządzania bazą danych wykorzystany do przechowywania danych aplikacji. [35]
- ASP.NET Core Identity — mechanizm odpowiedzialny za zarządzanie kontami użytkowników oraz uwierzytelnianie w aplikacji webowej. [36]
- OpenIddict — biblioteka wykorzystana do implementacji serwera OAuth 2.0/OpenID Connect oraz obsługi uwierzytelniania aplikacji desktopowej. [37]

- Tailwind CSS — framework CSS wykorzystany do tworzenia i stylowania interfejsu użytkownika. [38]

5.1.2. Aplikacja desktopowa

- Rust — język programowania wykorzystany do implementacji aplikacji desktopowej, w szczególności ze względu na wydajność oraz mechanizmy wspierające bezpieczne programowanie współbieżne. [39]
- Cargo — standardowy menedżer pakietów i system budowania projektów języka Rust, wykorzystany do zarządzania zależnościami oraz kompilowania aplikacji desktopowej. [40]
- Slint — framework wykorzystany do implementacji graficznego interfejsu użytkownika aplikacji desktopowej. [41]
- scap — biblioteka będąca wraperem dla Windows.Graphics.Capture, ScreenCaptureKit i Pipewire, wykorzystana do ciągłego przechwytywania obrazu ekranu i dostarczania kolejnych klatek do potoku przetwarzania. [42]
- CPAL (Cross-Platform Audio Library) — biblioteka wykorzystana do przechwytywania sampli dźwiękowych z urządzeń wejściowych i wyjściowych audio. [43]
- FFmpeg / ffmpeg-next — zestaw bibliotek multimedialnych oraz ich interfejs dla języka Rust, wykorzystane do przetwarzania, skalowania, enkodowania i multipleksowania strumieni audio i wideo. [44]
- rubato — biblioteka wykorzystana do resamplingu sygnału audio przed jego dalszym przetwarzaniem. [45]
- ringbuf — biblioteka implementująca ring buffer wykorzystany do buforowania danych audio pomiędzy etapami potoku. [46]
- crossbeam-channel — biblioteka zapewniająca kanały komunikacyjne wykorzystywane do wymiany danych pomiędzy współbieżnie działającymi częściami aplikacji. [47]
- Tokio — asynchroniczny runtime języka Rust wykorzystany przede wszystkim do obsługi operacji sieciowych. [48]
- Reqwest — biblioteka klienta HTTP wykorzystana do komunikacji aplikacji desktopowej z API aplikacji webowej. [49]
- Serde / serde_json — biblioteki wykorzystane do serializacji i deserializacji struktur danych, w szczególności podczas komunikacji z API. [50]
- uuid — biblioteka wykorzystana do generowania identyfikatorów oraz wartości stosowanych w mechanizmach wymagających unikalnych identyfikatorów. [51]

5.2. Implementacja potoku przechwytywania i przetwarzania obrazu

Jednym z podstawowych zadań aplikacji desktopowej jest przechwytywanie obrazu ekranu oraz przekształcenie otrzymanych danych do postaci umożliwiającej ich zapisanie w postaci skompresowanego strumienia wideo. Proces ten został zaimplementowany jako potok przetwarzania, w którym kolejne etapy odpowiadają za przechwycenie klatki, jej przetworzenie oraz przekazanie do enkodera.

W odróżnieniu od prostego zapisu kolejnych obrazów, przechwytywanie obrazu w czasie rzeczywistym wymaga przetwarzania kolejnych klatek z określoną częstotliwością. Każda klatka musi zostać odebrana, przetworzona i zakodowana w czasie krótszym od czasu dostępnego pomiędzy kolejnymi klatkami. Przekroczenie tego czasu może prowadzić do powstawania opóźnień lub pomijania części przechwyconych klatek. Z tego względu sposób organizacji poszczególnych etapów potoku oraz ich wydajność mają bezpośredni wpływ na płynność nagrywania.

W implementacji za przechwytywanie obrazu odpowiada biblioteka scap, natomiast dalsze operacje na danych obrazu, w tym ich konwersja i kodowanie, realizowane są z wykorzystaniem biblioteki FFmpeg poprzez interfejs ffmpeg-next.

5.2.1. Przechwytywanie klatek obrazu

Do przechwytywania obrazu wykorzystano bibliotekę scap, która udostępnia jednolity interfejs programistyczny do mechanizmów przechwytywania obrazu dostarczanych przez system operacyjny - `Windows.Graphics.Capture`, `ScreenCaptureKit` i `Pipewire`.[42] Zadaniem tego komponentu jest pozyskanie kolejnych klatek obrazu oraz przekazanie ich do dalszej części potoku.

Proces nagrywania rozpoczyna się od utworzenia konfiguracji przechwytywania oraz wskazania źródła obrazu. Po uruchomieniu przechwytywania aplikacja otrzymuje kolejne klatki w postaci danych obrazu. Klatka stanowi niezależny zestaw danych reprezentujący stan przechwytywanego obrazu w określonym momencie.

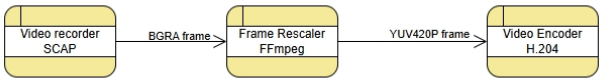
W tym miejscu istotne jest rozróżnienie pomiędzy przechwytywaniem obrazu, a jego enkodowaniem. Biblioteka scap odpowiada wyłącznie za dostarczenie surowych ramek danych obrazu. Otrzymana klatka nie jest jeszcze skompresowanym materiałem wideo i musi najpierw zostać odpowiednio przygotowana, a następnie przekazana do enkodera wideo. W implementacji przechwycone klatki są przekazywane do dalszych etapów potoku bez wykonywania procesu enkodowania bezpośrednio w mechanizmie odpowiedzialnym za przechwytywanie. Takie rozdzielenie odpowiedzialności pozwala oddzielić zależne od systemu operacyjnego pozyskiwanie obrazu od operacji związanych z jego przetwarzaniem i enkodowaniem.

5.2.2. Przygotowanie klatek do enkodowania

Dane otrzymane z mechanizmu przechwytywania muszą zostać doprowadzone do formatu akceptowanego przez wykorzystywany enkoder. W szczególności dotyczy to formatu próbek obrazu oraz jego rozdzielczości.

W implementacji przechwycone klatki są reprezentowane w formacie BGRA. Jest to format wykorzystujący cztery składowe dla każdego piksela: niebieską, zieloną, czerwoną oraz kanał alfa. Format ten jest dogodny na etapie przechwytywania obrazu, jednak nie jest formatem wykorzystywanym przez zastosowany enkoder H.264 – YUV420p.

Dlatego pomiędzy przechwytywaniem, a enkodowaniem zastosowano etap konwersji obrazu realizowany przy użyciu mechanizmu skalowania dostarczanego przez FFmpeg. Odpowiada on jednocześnie za ewentualną zmianę rozdzielczości w przypadku przełączenia źródła przechwytywanych ramek obrazu oraz konwersję formatu pikseli. (rys. 5.1)



Rysunek 5.1: Diagram przepływu przechwyconych ramek video

Zastosowanie osobnego etapu konwersji pozwala uniezależnić format danych dostarczanych przez mechanizm przechwytywania od wymagań dalszego przetwarzania. Jest to szczególnie istotne w przypadku wykorzystania bibliotek systemowych do przechwytywania obrazu, ponieważ format otrzymywanych danych może wynikać z konfiguracji konkretnego mechanizmu przechwytywania, podczas gdy enkoder wymaga określonego formatu wejściowego.

Konwersja wykonywana jest za pomocą obiektu software::scaling::Context biblioteki FFmpeg. Obiekt ten przechowuje konfigurację operacji przekształcania obrazu i jest wykorzystywany dla kolejnych przechwyconych klatek.

Takie rozwiązanie pozwala również na zmianę rozdzielczości nagrywanego obrazu. Operacja ta ma istotne znaczenie z punktu widzenia wydajności, ponieważ liczba pikseli przypadających na pojedynczą klatkę bezpośrednio wpływa na ilość danych wymagających przetworzenia. Przykładowo, zwiększenie rozdzielczości z 1920×1080 do 3840×2160 powoduje czterokrotny wzrost liczby pikseli przypadających na pojedynczą klatkę.

W związku z tym skalowanie obrazu stanowi nie tylko operację techniczną wymaganą przez enkoder, ale również jeden z mechanizmów pozwalających kontrolować koszt obliczeniowy całego procesu nagrywania.

5.2.3. Kodowanie strumienia video

Po przygotowaniu klatki jest ona przekazywana do enkodera wideo FFmpeg. W implementacji wykorzystano kodek H.264, który pozwala znacząco zmniejszyć rozmiar danych względem nieskompresowanego obrazu przy zachowaniu jakości odpowiedniej dla nagrań ekranu.

Enkoder otrzymuje kolejne klatki w formacie przygotowanym przez Frame Rescaler (rys. 5.1), ustawia presentation timestamp[52] ramki i przekształca je w zakodowane pakiety wideo. Pakiety te nie stanowią jeszcze kompletnego pliku wideo. Są przekazywane do multipleksera, odpowiedzialnego za połączenie zakodowanych danych obrazu i dźwięku oraz zapisanie ich w kontenerze.

Konfiguracja enkodera obejmuje między innymi rozdzielczość obrazu, częstotliwość klatek, format pikseli, bitrate oraz parametry wpływające na złożoność procesu kodowania. W projekcie wykorzystano tryby ultrafast oraz Constant Rate Factor, które pozwalają na zachowanie kompromisu pomiędzy szybkością kodowania, a uzyskiwaną jakością i rozmiarem materiału.

Istotnym parametrem jest również liczba klatek typu B – (bi-directional predictive frame) – klatek kompresji tworzonych na podstawie poprzedzających i następującej po niej klatkach. W implementacji ich wykorzystanie zostało wyłączone, co upraszcza zależności pomiędzy kolejnymi klatkami i ogranicza wymagania związane z buforowaniem podczas kodowania. Jest to korzystne w przypadku przetwarzania materiału w czasie rzeczywistym, gdzie istotne jest ograniczenie opóźnienia pomiędzy przechwyceniem obrazu a uzyskaniem zakodowanego pakietu.

Po zakodowaniu klatki wynikowy AVPacket jest przekazywany do dalszej części potoku. Na tym etapie dane obrazu są już skompresowane, jednak nadal reprezentują wyłącznie strumień wideo. Połączenie go ze strumieniem audio oraz zapis w wybranym kontenerze odbywa się w późniejszym etapie procesu nagrywania, opisanym w dalszej części rozdziału. 5.4.

5.3. Implementacja potoku przetwarzania dźwięku

Drugim strumieniem danych przetwarzanym przez aplikację desktopową jest dźwięk. W przeciwieństwie do obrazu, którego kolejne klatki są dostarczane przez mechanizm przechwytywania ekranu, dane audio są pozyskiwane z niezależnych źródeł wejściowych, a następnie przekazywane do aplikacji w postaci próbek sygnału.

Oba źródła są przechwytywane niezależnie, dlatego ich dane nie tworzą od początku jednego strumienia audio. Przed zakodowaniem konieczne jest ich odpowiednie zgromadzenie, zsynchronizowanie w ramach dostępnych porcji danych oraz zmiksowanie do wspólnego strumienia. Dopiero tak przygotowany sygnał jest przekazywany do dalszego przetwarzania i zakodowania.

Za przechwytywanie danych audio odpowiada biblioteka CPAL, natomiast do buforowania wykorzystano ring buffer. Miksowanie i przygotowanie danych realizowane jest w kodzie aplikacji, a operacje związane z resamplingiem i enkodowaniem wykonywane są z wykorzystaniem bibliotek rubato oraz FFmpeg.

Potok audio został podzielony na kilka etapów: przechwytywanie próbek, ich buforowanie, resampling, miksowanie, enkodowanie oraz przekazanie zakodowanych danych do dalszego etapu procesu nagrywania. (rys. 5.2)

Do obsługi urządzeń audio wykorzystano bibliotekę CPAL (Cross-Platform Audio Library)[43]. Zapewnia ona abstrakcję nad systemowymi mechanizmami obsługi dźwięku i umożliwia tworzenie strumieni audio dostarczających aplikacji kolejne próbki sygnału.

W systemie zaimplementowano możliwość przechwytywania dwóch niezależnych źródeł audio. Pierwszym jest mikrofon, którego zadaniem jest dostarczanie głosu użytkownika. Drugim jest dźwięk systemowy, zawierający między innymi dźwięk odtwarzanych aplikacji. Każde ze źródeł jest obsługiwane jako osobny strumień danych.

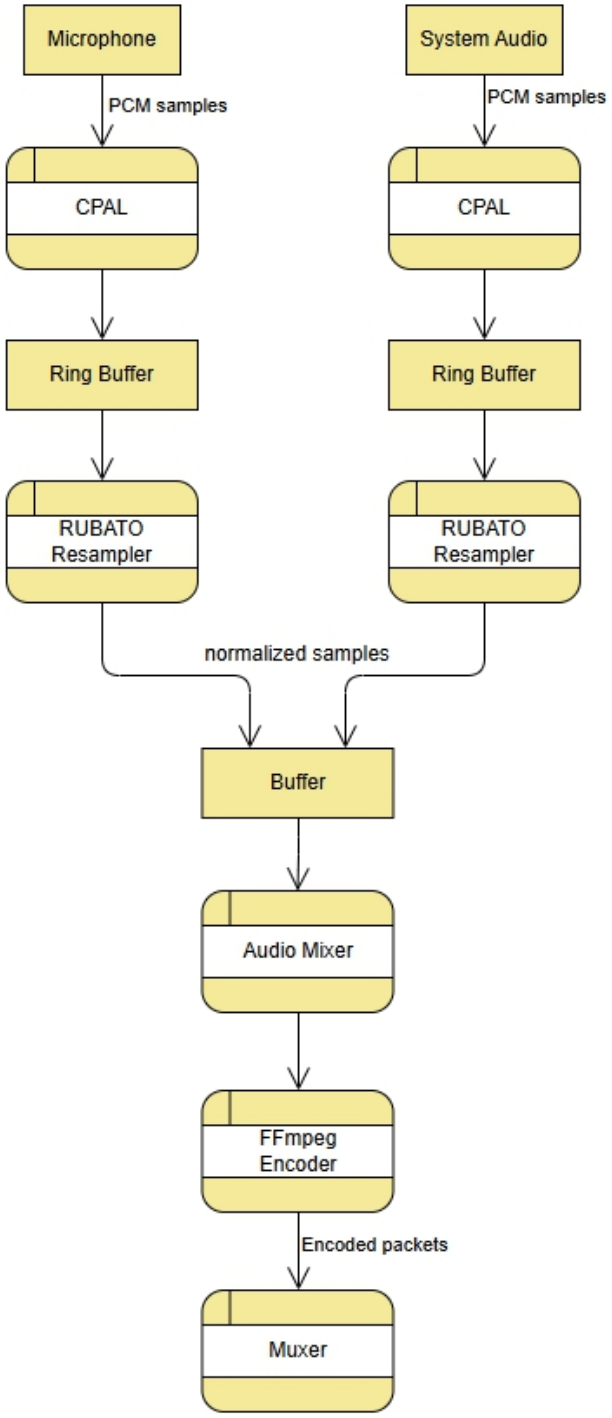
Takie rozwiązanie pozwala na niezależne pozyskiwanie obu ścieżek dźwiękowych i ich późniejsze połączenie. Jest to istotne również ze względu na możliwość występowania różnic pomiędzy źródłami, takich jak częstotliwość próbkowania, liczba kanałów czy wielkość kolejnych sampli.

Odbiór danych z urządzeń audio realizowany jest za pomocą mechanizmu callbacków dostarczanych przez CPAL. Callback jest wywoływany w momencie udostępnienia przez system kolejnej porcji sampli audio. Ze względu na charakter operacji, jego wykonanie powinno być możliwie krótkie. Umieszczanie w nim kosztownych operacji przetwarzania mogłoby doprowadzić do opóźnienia obsługi kolejnych porcji danych i w konsekwencji do utraty sampli.

Z tego względu callbacki pełnią przede wszystkim rolę producentów danych dla dalszego potoku. Otrzymane próbki są przekazywane do buforów, a właściwe przetwarzanie odbywa się poza mechanizmem odpowiedzialnym bezpośrednio za odbiór danych z urządzenia

5.3.1. Buforowanie sampli audio

Dane przechwycone z mikrofonu i dźwięku systemowego są przechowywane w niezależnych buforach cyklicznych(ring buffers). Wykorzystanie biblioteki ringbuf umożliwia oddzielenie momentu dostarczenia próbek przez źródło audio od momentu ich pobrania przez część aplikacji odpowiedzialną za dalsze przetwarzanie. Każdy ze strumieni jest traktowany jako niezależny producent danych. Buforowanie ma szczególne znaczenie w przypadku dwóch źródeł, ponieważ nie ma gwarancji, że oba źródła będą dostarczały dane jednocześnie oraz w identycznych porcjach. Zastoso-



Rysunek 5.2: Diagram przepływu przechwyconych sampli audio

wanie buforów stanowi jednocześnie mechanizm izolujący czasowo przechwytywanie od dalszego przetwarzania. Krótkotrwałe różnice w czasie wykonania poszczególnych etapów mogą dzięki temu zostać absorbowane bez konieczności blokowania mechanizmu przechwytywania. Pojemność bufora jest jednak ograniczona. Jeżeli dalsze przetwarzanie będzie przez dłuższy czas wolniejsze od tempa dostarczania danych, bufor może się zapełnić, co może prowadzić do utraty kolejnych sampli. Problem ten jest szczególnie istotny dla systemów czasu rzeczywistego i zostanie szerzej omówiony w części poświęconej współbieżności oraz ograniczeniom wydajnościowym.5.5.3.

5.3.2. Resampling sygnału

Dane dostarczane przez urządzenie audio nie muszą posiadać parametrów odpowiadających konfiguracji przyjętej przez enkoder. Jednym z parametrów, który może wymagać dostosowania, jest częstotliwość próbkowania.

Do realizacji resamplingu wykorzystano bibliotekę rubato. Resampling polega na zmianie częstotliwości próbkowania sygnału przy zachowaniu jego reprezentacji w dziedzinie czasu.

Operacja jest wykonywana na porcjach próbek pobieranych z potoku audio. Pozwala to przenieść koszt obliczeniowy resamplingu poza callbacki urządzeń audio i wykonywać go w ramach właściwego etapu przetwarzania.

W rezultacie przed przekazaniem danych do miksera uzyskiwany jest strumień o ujednoliconych parametrach, co upraszcza realizację kolejnego etapu.

5.3.3. Miksowanie sampli

Po przechwyceniu i znormalizowaniu danych z obu źródeł kolejne porcje sampli są łączone w jeden strumień audio. Proces ten realizuje komponent odpowiedzialny za miksowanie sampli. W najprostszym przypadku wartość sampla wynikowego może zostać określona jako suma odpowiadających sobie próbek obu źródeł:
`y[i] = x_m[i] + x_s[i]` gdzie:
`x_m[i]` oznacza sample pochodzącą z mikrofonu,
`x_s[i]` sample dźwięku systemowego,
natomiast `y[i]` odpowiadający im element wynikowego strumienia.

Bezpośrednie dodawanie próbek nie zawsze jest jednak wystarczające. W zależności od reprezentacji sygnału suma może przekroczyć dopuszczalny zakres wartości i doprowadzić do zniekształcenia sygnału. Z tego względu implementacja miksera uwzględnia sposób reprezentacji próbek oraz zakres wartości obsługiwany przez dalsze etapy przetwarzania oraz dokonuje normalizacji względem najwyższej wartości bezwzględnej zarejestrowanego sygnału.

Synchronizacja strumieni audio

Miksowanie pełni również funkcję pośredniego etapu synchronizującego oba źródła. Mixer gorliwie pobiera dane z odpowiednich buforów i tworzy z nich kolejne porcje wspólnego strumienia. W przypadku gdy system nie odtwarza żadnego dźwięku, buffor do niego przypisany nie jest wypełniany. Obecna implementacja rozwiązuje problem synchronizacji w trywialny sposób – w momencie dostępności sampli tylko z jednego źródła, brakująca część sampli z drugiego wypełniana jest sztucznie generowaną ciszą. Oczywiście jest to źródłem problemów, które zostaną opisane w dalszej części pracy. 5.3.3.

Od tego momentu dalsza część potoku przetwarzania nie musi rozróżniać źródła sampli.

Jest to istotna cecha przyjętego rozwiązania. Zamiast enkodować oba źródła jako dwa niezależne strumienie audio, zostały one połączone przed enkoderem, dzięki czemu wynikowy plik zawiera pojedynczy strumień audio zawierający oba źródła dźwięku.

Problem synchronizacji strumieni audio

Ze względu na specyfikę obecnej implementacji mixowania audio opisanej w sekcji 5.3.3., może dojść do następującej sytuacji:

```
Bufor1:<M0, M1,..., Mi,..., Mn, Mn+1, Mn+2, ..., M2n>
Bufor2:             <S0,..., Sj, Sj+1, ..., Sn, Sn+1, ...>
        T0, T1,..., Ti,....
```

gdzie:
M_i oznacza i-tą próbkę sygnału audio źródła pierwszego
S_j – j-tą próbkę sygnału audio źródła drugiego
Ti – moment i
oraz `i,j` $\in$ `<0,n>`

tj. źródło 2 zaczęło nadawać sygnał w momencie, gdy bufor źródła 2 miał wymaganą liczbę sampli do zmiksowania Wtedy wynikiem zmiksowanego sygnału będzie $M_0, M_1, ..., M_n, M_{n+1} + S_0, M_{n+2} + S_1...$ – dojdzie do desynchronizacji ścieżek audio w czasie.

Trywialny sposób rozwiązania - odrzucenie spóźnionych do przetwarzania sampli generuje artefakty audio. Właściwym rozwiązaniem jest implementacja mechanizmu synchronizacji względem zewnętrznego zegara.

5.3.4. Enkodowanie dźwięku

Ostatnim etapem potoku audio przed muxowaniem jest kodowanie przygotowanych próbek. Do tego celu wykorzystano bibliotekę FFmpeg poprzez interfejs ffmpeg-next.

Enkoder otrzymuje kolejne porcje przygotowanych próbek audio i przekształca je w skompresowane pakiety. Pakiety te posiadają informacje niezbędne do ich późniejszego umieszczenia w kontenerze multimedialnym, w szczególności związane z czasem prezentacji danych.

Podobnie jak w przypadku strumienia wideo, etap kodowania jest oddzielony od muxowania. Oznacza to, że enkoder audio nie tworzy samodzielnie pliku końcowego. Jego zadaniem jest jedynie przygotowanie zakodowanego strumienia, który następnie zostanie połączony ze strumieniem wideo.

5.4. Multipleksowanie, synchronizacja i zapis do pliku

Po zakończeniu niezależnego przetwarzania obrazu i dźwięku oba strumienie muszą zostać połączone w jeden materiał multimedialny. Zadanie to realizowane jest przez multiplexer FFmpeg. W odróżnieniu od enkodera, którego zadaniem jest kompresja danych, multiplexer odpowiada za umieszczenie zakodowanych strumieni w wybranym kontenerze multimedialnym.

W implementacji wykorzystano kontener MP4. Proces zapisu rozpoczyna się od utworzenia kontekstu formatu oraz skonfigurowania osobnych strumieni dla obrazu i dźwięku. Enkodery audio i video przekazują następnie do muxera kolejne zakodowane pakiety w dowolnej kolejności.

5.4.1. Znaczniki czasowe i synchronizacja

Istotnym elementem zapisu materiału multimedialnego są znaczniki czasowe pakietów. Obraz i dźwięk są przetwarzane niezależnie, dlatego kolejność otrzymywania pakietów przez muxer nie musi odpowiadać bezpośrednio ich kolejności czasowej.

Każdy ze strumieni posiada własną podstawę czasu (time base), względem której określane są wartości PTS (Presentation Time Stamp). Informacje te pozwalają odtworzyć właściwą relację czasową pomiędzy danymi audio i video.

W praktyce oznacza to, że multipleksowanie nie polega jedynie na naprzemiennym zapisywaniu pakietów audio i video. Pakiety muszą zostać opatrzone odpowiednimi informacjami czasowymi, a następnie zapisane w sposób zgodny z wymaganiami kontenera.

Jest to szczególnie istotne w przypadku współbieżnego przetwarzania, ponieważ

pakiety mogą być generowane w różnym tempie. Prawidłowe wykorzystanie znaczników czasowych pozwala oddzielić kolejność wykonania operacji w programie od czasu prezentacji danych podczas późniejszego odtwarzania.

5.4.2. Zakończenie zapisu

Zakończenie nagrywania wymaga opróżnienia buforów oraz enkoderów. Samo zatrzymanie przechwytywania nie oznacza bowiem, że wszystkie dane zostały już zapisane do pliku.

Enkodery mogą przechowywać dane oczekujące na wygenerowanie kolejnych pakietów. Z tego względu proces kończenia sesji nagrywania obejmuje zatrzymanie źródeł danych, obsługę pozostałych danych znajdujących się w potoku oraz finalizację muxera.

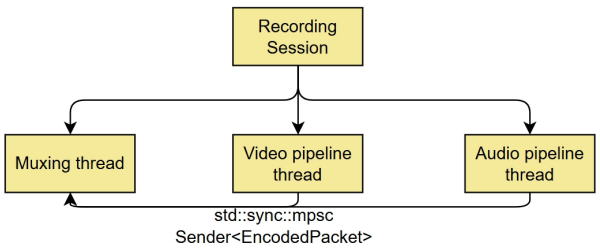
Dopiero po wykonaniu tych operacji plik może zostać prawidłowo zamknięty. Ma to znaczenie szczególnie w przypadku kontenerów posiadających strukturę wymagającą zapisania informacji końcowych.

Ostatecznie otrzymywany jest plik MP4 zawierający pojedynczy strumień audio, powstały z połączenia dźwięku wybranych źródeł, oraz strumień video zawierający przechwycony obraz ekranu.

5.5. Współbieżne przetwarzanie strumieni

Przetwarzanie materiału multimedialnego w czasie rzeczywistym wymaga właściwej organizacji wykonywania poszczególnych operacji. Dane są dostarczane w sposób ciągły, dlatego operacje nie mogą blokować się wzajemnie.

W implementacji zastosowano podział procesu nagrywania na trzy główne obszary wykonywane współbieżnie: przetwarzanie dźwięku, przetwarzanie obrazu oraz multipleksowanie. Każdy z nich posiada własny wątek wykonawczy. Dodatkowo mechanizmy przechwytywania audio korzystają z callbacków strumieni CPAL, które działają niezależnie od głównego wątku przetwarzania audio.



Rysunek 5.3: Diagram organizacji współbieżnego przetwarzania strumieni

5.5.1. Organizacja współbieżnego potoku

Koordynatorem procesu nagrywania jest sesja nagrywania (RecordingSession). Inicjalizuje on mupltiplekser, koordynuje uruchamianie i zatrzymywanie poszczególnych elementów procesu oraz zapewnia przekazywanie kontekstu pomiędzy nimi. (rys. 5.3)

Komponenty wykonujące operacje związane z danym rodzajem danych zostały zgrupowane w ramach jednego wątku. Wątek audio obejmuje pobieranie sampli, resampling, miksowanie oraz enkodowanie. Analogicznie wątek video obsługuje przechwytywanie i obróbkę oraz enkodowanie klatek.

W przypadku audio biblioteka CPAL dostarcza dane za pomocą callbacków strumieni audio. Callback przekazuje kolejne sample do struktury ring buffer, z których są następnie pobierane przez właściwy wątek audio.

W przypadku obrazu mechanizm przechwytywania jest wykorzystywany wewnątrz wątku video. Oznacza to, że zarówno przechwytywanie kolejnych klatek, jak i ich przygotowanie oraz enkodowanie odbywają się w ramach tego samego głównego wątku potoku video.

Takie zgrupowanie operacji ogranicza liczbę tworzonych wątków, a jednocześnie pozwala niezależnie wykonywać kosztowne potoki audio i video.

5.5.2. Komunikacja pomiędzy potokami

Do komunikacji pomiędzy głównymi wątkami wykorzystano kanały std::sync::mpsc (multi-producer, single-consumer), będące częścią standardowej biblioteki języka Rust. Najważniejszym kanałem jest kanał przeznaczony do przesyłania obiektów EncodedPacket. Jest on tworzony przez muxer, a jego transmitter przekazywany jest zarówno do enkodera audio, jak i enkodera video. W rezultacie oba niezależne potoki mogą przekazywać gotowe pakiety do jednego odbiorcy – multipleksera. (rys. 5.3)

Zastosowanie jednego kanału dla obu rodzajów zakodowanych danych pozwala oddzielić generowanie pakietów od ich zapisywania. Enkoder nie musi bezpośrednio wykonywać operacji zapisu do pliku, a muxer nie musi wykonywać operacji związanych z przechwytywaniem lub kodowaniem.

Muxer odbiera kolejne pakiety z kanału i przekazuje je do mechanizmu zapisu FFmpeg. W jego wątku wykonywane jest oczekiwanie na kolejne dane za pomocą operacji recv(), a otrzymane pakiety są następnie zapisywane do kontenera.

Kanały mpsc są wykorzystywane również do innych celów związanych z koordynacją sesji. Przykładowo, RecordingSession wykorzystuje kanał do przekazywania poleceń do wątku video oraz kanał synchronizacyjny do odbierania informacji o stanie inicjalizacji enkoderów.

5.5.3. Problem różnicy szybkości przetwarzania na przykładzie wideo

Wprowadzenie niezależnych wątków rozwiązuje problem wzajemnego blokowania się głównych potoków, ale nie eliminuje problemu różnicy pomiędzy szybkością dostarczania danych a szybkością ich przetwarzania.

Jeżeli czas potrzebny na przetworzenie kolejnej klatki jest zbyt duży w stosunku do częstotliwości przechwytywania, system nie będzie w stanie utrzymać zadanej liczby klatek na sekundę.

Przykładowo dla częstotliwości 60 FPS średni czas przypadający na jedną klatkę wynosi: $T_{frame} = \frac{1}{60} \approx 16{,}67\ ms$
Wartość ta stanowi punkt odniesienia dla przepustowości potoku przechwytywania obrazu.

Jeżeli kolejne etapy przetwarzania wymagają średnio więcej czasu, niż wynika to z częstotliwości dostarczania danych, pojawia się narastające opóźnienie lub konieczność pomijania części danych.

W przypadku obrazu problem ten jest szczególnie widoczny przy zwiększaniu rozdzielczości. Liczba pikseli przetwarzanych w pojedynczej klatce rośnie wraz z kwadratem liniowego zwiększenia rozdzielczości. Przejście z 1920×1080 do 3840×2160 zwiększa liczbę pikseli czterokrotnie:
$1920 \cdot 1080 = 2,073,600$
$3840 \cdot 2160 = 8,294,400$

W rezultacie zwiększenie rozdzielczości może spowodować przekroczenie możliwości obliczeniowych procesora i prowadzić do spadku rzeczywistej liczby klatek lub wzrostu opóźnienia.

5.5.4. Buforowanie i utrata danych

Bufory stanowią niezbędny element potoku czasu rzeczywistego, ale jednocześnie wprowadzają kompromis pomiędzy odpornością na chwilowe przeciążenie a zużyciem pamięci.

Zbyt mały bufor może szybko się przepełnić, jeżeli konsument chwilowo nie nadąża za producentem. Zbyt duży lub nieograniczony bufor pozwala natomiast akumulować coraz większą ilość danych, co może prowadzić do znacznego wzrostu zużycia pamięci.

W przypadku nagrywania obrazu problem ten jest szczególnie widoczny, ponieważ pojedyncza nieskompresowana klatka może zajmować znaczną ilość pamięci. Przy zwiększaniu rozdzielczości nawet niewielka liczba zalegających klatek może zatem powodować zauważalny wzrost wykorzystania pamięci RAM.

5.5.5. Ograniczenia programowego enkodowania obrazu

W obecnej implementacji enkodowanie wideo realizowane jest programowo z wykorzystaniem FFmpeg, co oznacza, że operacje te wykonywane są z wykorzystaniem zasobów procesora.

Rozwiązanie to znacząco upraszcza implementację i nie wymaga uzależnienia aplikacji od konkretnego producenta układu graficznego. Jednocześnie jego wydajność jest ograniczona dostępną mocą obliczeniową CPU.

Przy odpowiednio dużej rozdzielczości lub liczbie klatek proces enkodowania może znacznie ograniczyć przpustowość całego potoku. W takiej sytuacji samo zwiększanie rozmiaru buforów nie rozwiązuje problemu, ponieważ powoduje akumulowanie coraz większej liczby danych oczekujących na przetworzenie.

Alternatywą jest zastosowanie akceleracji sprzętowej, która umożliwia odciążenie CPU. Wprowadzenie sprzętowego enkodera nie jest jednak wyłącznie zamianą jednego komponentu na inny. W zależności od zastosowanego rozwiązania może być konieczne uwzględnienie sposobu przekazywania klatek pomiędzy pamięcią systemową, a pamięcią GPU oraz obsługi konkretnego interfejsu sprzętowego.

Z tego względu obecna implementacja wykorzystująca programowe enkodowanie stanowi zadowalające rozwiązanie na potrzeby realizacji tej pracy, natomiast akceleracja sprzętowa jest jednym z możliwych kierunków dalszej optymalizacji.

5.6. Implementacja przesyłania plików

Przesyłanie nagrania stanowi podstawowy mechanizm komunikacji pomiędzy aplikacją desktopową a serwerem. Implementacja została zaprojektowana z uwzględnieniem faktu, że nagrania wideo mogą posiadać znaczny rozmiar, dlatego przesłanie całego pliku jako pojedynczego żądania nie jest jedynym możliwym sposobem realizacji tej operacji.

Po stronie serwera obsługę procesu powierzono komponentowi UploadService. Odpowiada on między innymi za utworzenie rekordu uploadu, przygotowanie tymczasowego miejsca przechowywania danych oraz aktualizację jego stanu.

5.6.1. Upload dzielony na fragmenty

Jednym z zaimplementowanych mechanizmów jest przesyłanie pliku w postaci niezależnych fragmentów przesyłanych równolegle. Przed rozpoczęciem transferu klient przekazuje informacje dotyczące pliku oraz parametrów uploadu. Serwer tworzy wtedy obiekt reprezentujący rozpoczęte przesyłanie.

Kolejne fragmenty są przesyłane za pomocą osobnych żądań HTTP. Każdy fragment jest identyfikowany poprzez numer odpowiadający jego pozycji w pliku.

Takie rozwiązanie pozwala ograniczyć rozmiar pojedynczego żądania oraz umożliwia śledzenie postępu przesyłania. Serwer przechowuje stan uploadu, dzięki czemu możliwe jest określenie liczby otrzymanych fragmentów oraz ilości przesłanych danych.

5.6.2. Przechowywanie pliku tymczasowego

Podczas przesyłania dane nie są od razu traktowane jako gotowe nagranie. Są zapisywane w tymczasowym pliku przypisanym do konkretnego uploadu.

Rozdzielenie tymczasowego przechowywania od właściwego zasobu video pozwala uniknąć sytuacji, w której niepełny lub przerwany upload byłby prezentowany użytkownikowi jako poprawne nagranie.

Po zakończeniu transferu plik może zostać przekazany do kolejnego etapu obsługi materiału.

5.6.3. Kontrola stanu uploadu

API udostępnia również możliwość odczytania stanu rozpoczętego przesyłania. Zwracane informacje obejmują między innymi rozmiar pliku, liczbę przesłanych danych, liczbę fragmentów oraz informacje dotyczące rozpoczęcia i zakończenia procesu.

Takie podejście pozwala aplikacji klienckiej monitorować postęp operacji oraz odpowiednio reagować na sytuacje, w których przesyłanie nie zostało jeszcze zakończone.

Implementacja wykorzystuje zatem standardowe mechanizmy HTTP i ASP.NET Core, natomiast istotnym elementem własnego rozwiązania jest organizacja uploadu jako procesu posiadającego własny stan oraz możliwość obsługi danych przesyłanych fragmentami.

5.7. Rozszerzalność implementacji

Jednym z celów przyjętej implementacji było ograniczenie zależności pomiędzy poszczególnymi etapami przetwarzania. Zastosowanie osobnych komponentów dla przechwytywania, przygotowania danych, enkodowania i multipleksowania pozwala modyfikować poszczególne etapy bez konieczności przebudowy całego procesu.

Analogiczny mechanizm zastosowano po stronie serwera poprzez interfejs IVideoProcessor. Obecna implementacja symuluje przetwarzanie materiału, ale sposób jej

wdrożenia do procesu obsługi nagrania pozwala w przyszłości zastąpić ją właściwym mechanizmem przetwarzania wykorzystującym FFmpeg.

Takie rozwiązanie pozwala traktować obecną implementację jako działający fundament dalszego rozwoju systemu, a jednocześnie umożliwia niezależne rozwijanie najbardziej kosztownych elementów, takich jak rzeczywiste przetwarzanie materiału po stronie serwera czy sprzętowe enkodowanie obrazu.

Rozdział 6.

Część dla programisty

Pobranie kodu źródłowego projektu

Kod źródłowy projektu dostępny jest w prywatnym repozytorium pod adresem `https://github.com/MADkarmi/ScreenCap/`. By uzyskać dostęp, należy przesłać prośbę do autora pracy na adres radoslawmadzia@gmail.com

Struktura repozytorium

```
ScreenCap
        Desktop_App_Latest_Release_Windows_X86_64
        README.md
        scrap-desktop-app < Folder projektu aplikacji desktopowej
            README.md
        scrap-web-app
            Scrap-web
                Readme.md
                Scrap-web < Folder projektu aplikacji webowej
                Scrap-web.AppHost < future feature
                Scrap-web.ServiceDefaults < future feature
                Scrap-web.sln
```

W folderze `scrap-desktop-ap` znajduje się kod źródłowy aplikacji desktopowej.

W folderze `Scrap-web` znajduje się kod źródłowy aplikacji webowej.

W repozytorium umieszczono również plik wykonywalny najnowszej wersji aplikacji desktopowej wraz z wymaganymi zależnościami.

6.1. Przygotowanie środowiska i kompilacja

6.1.1. Aplikacja webowa

Przygotowanie środowiska

Aktualna wersja oparta jest na wersji`8 .NET`, której wsparcie kończy się 10 Listopada 2026 [28] Kolejna planowana wersja zostanie przeniesiona na `.NET 10`.

Aktualnie wymagane są:

1. `.NET 8 SDK`
2. `SQL Server\SQL Server LocalDB`
3. `Installing Entity Framework Core`
4. Dowolna przeglądarka wspierająca HTML5

Kompilacja i uruchomienie

Następujące instrukcje należy wykonywać z poziomu korzenia folderu projektu: `Scrap-web`

1. Upewnić się, że `Entity Framework Core` jest zainstalowany
   `dotnet tool list --global`
2. Przygotować certyfikat dla środowiska deweloperskiego
   `dotnet dev-certs https --check`.
   Jeżeli nie jest zaufany
   `dotnet dev-certs https --trust`
3. `dotnet restore`
4. Opcjonalnie można skonfigurować porty w plikach `appsettings.json` oraz `launchSettings.json`
5. `dotnet buid`
6. `dotnet-ef database update`
7. `dotnet run --launch-profile https`

Bądź otworzyć plik `Scrap-web.sln` przy użyciu `Visual Studio 2026 lub 2022`, upewnić się, że projekt startowy ustawiony jest na `Scrap-web` i odtworzyć bazę danych przez polecenie `dotnet-ef database update`

6.1.2. Aplikacja desktopowa

Przygotowanie środowiska

Aktualne instrukcje przygotowania środowiska znajdują się również w pliku *README.md* umieszczonym w korzeniu drzewka folderów projektu.

W przypadku systemu operacyjnego Windows, wspierana jest wersja 10 i 11.

Windows - architektura X86-64

Wymagany jest `LLVM` oraz dodana ścieżką folderu `bin` do zmiennej środowiskowej `PATH` bądź pod utworzoną zmienną `LIBCLANG_PATH`

Wymagany jest `FFmpeg` w wersji `8.0` z plikami nagłówkowymi, dodanie zmiennej środowiskowej `FFMPEG_DIR` wskazującej na folder zawierający podfoldery `bin` i `include` oraz dodanie ścieżki folderu `bin` do zmiennej środowiskowej `PATH`

Można to zrealizować międzyinnymi na dwa poniższe sposoby:

- Metoda pierwsza
  1. Zainstalować `LLVM` przy użyciu
     `winget install -e --id LLVM.clangd`
  2. Zainstalować `FFmpeg` przy użyciu
     `winget install -e --id Gyan.FFmpeg.Shared`
- Metoda druga
  1. Zainstalować *vcpkg*
     `https://github.com/microsoft/vcpkg?tab=readme-ov-file#quick-start-windows`
  2. Dodać zmienną środowiskową `VCPKG_ROOT` by wskazywała na folder, w którym zainstalowano `vcpkg`
  3. Zainstalować *clang*
     `vcpkg install llvm[clang,target-x86]:x64-windows`
  4. Dodać zmienną środowiskową `LIBCLANG_PATH` by wskazywała na folder, w którym go zainstalowano
     `%VCPKG_ROOT%\installed\x64-windows\bin`
  5. Zainstalować *FFmpeg*
     `vcpkg install ffmpeg --triplet x64-windows`
  6. Dodać `%VCPKG_ROOT%\installed\x64-windows\bin` do zmiennej środowiskowej `PATH`

Linux

W przypadku pracy na systemie Linux, należy zainstalować `ffmpeg` oraz `alsa`. Na przykładzie systemów opartych na Debianie

```
sudo apt-get install clang libavcodec-dev libavformat-dev
    libavutil-dev libavfilter-dev libavdevice-dev
    libasound2-dev pkg-config
```

macOS

```
brew install pkg-config ffmpeg
```

Powyższe wymagania wynikają bezpośrednio z wykorzystania biblioteki `ffmpeg_next` `https://github.com/zmwangx/rust-ffmpeg/wiki/Notes-on-building`

Wymagana jest również instalacja środowiska Rust – `https://doc.rust-lang.org/book/ch01-01-installation.html`

Ze względu na zależności od aktywnie rozwijanych zewnętrznych bibliotek mogą pojawić się błędy kompilacji. Działająca instancja aplikacji oparta jest na kompilacie FFmpeg w wersji 8.0 z dnia 2025-11-17 `https://github.com/GyanD/codexffmpeg/releases/tag/2025-11-17-git-e94439e49b` - FFmpeg 8.0 "Huffman"[29]

Kompilacja i uruchomienie

Następujące instrukcje należy wykonywać z poziomu korzenia folderu projektu: `scrap-desktop-app`.

Jeżeli zostały dokonane zmiany portów w plikach `appsettings.json` oraz `launchSettings.json`, należy odpowiednio zmienić numery portów w pliku `.env`.

1. `cargo build`
2. `cargo run` można również uruchomić aplikację z odpowiednimi flagami wymienionymi w sekcji `[features]` pliku `Cargo.toml`

6.2. Metryka kodu

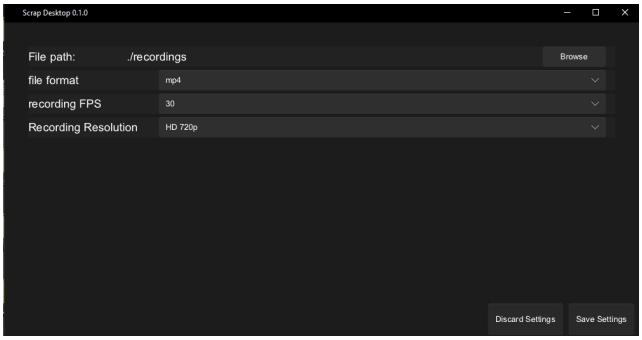
Aplikacja	Język programowania	Ilość linii
Web	C# + Razor	2713
Desktop	Rust	3409
Desktop	Slint	1047
Łącznie		7169

Rozdział 7.

Instrukcja użytkownika

7.1. Zmiana podstawowych parametrów przechwytywania obrazu

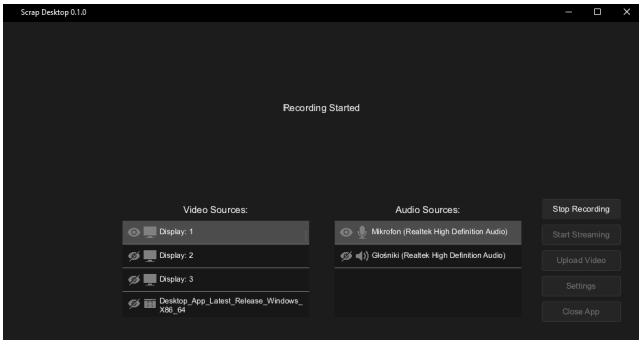
Po uruchomieniu aplikacji desktopowej użytkownik ma możliwość zmiany podstawowych ustawień przechwytywania oraz wyboru folderu w którym znajdzie się plik wynikowy. (rys. 7.1)



Rysunek 7.1: Widok zmiany podstawowych parametrów przechwytywania obrazu

7.2. Wybór źródeł przechwytywanych danych i rozpoczęcie nagrywania

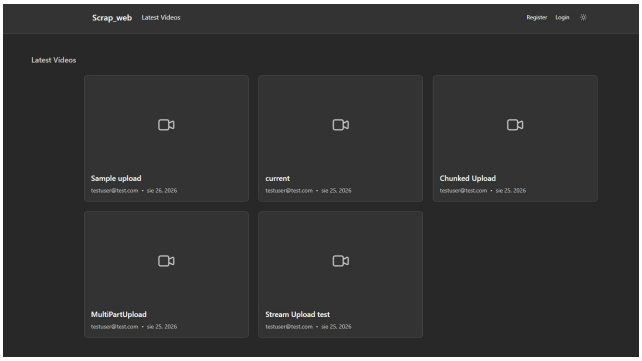
Chcąc rozpocząć nagrywanie użytownik zobowiązany jest do wyboru źródła obrazu. Wybrany podmiot został jednoznacznie oznaczony podświetleniem. Proces rejestracji oznaczony jest przez zmianę tekstu przycisku odpowiedzialnego za rozpoczęcie nagrywania oraz dezaktywację możliwości interakcji z pozostałymi elementami interfejsu użytkownika. (rys. 7.2)



Rysunek 7.2: Proces aktywnego rejestrowania obrazu

7.3. Przeglądanie plików dostępnych w aplikacji webowej

Niezalogowany użytkownik ma dostęp do publicznie udostępnionych materiałów. (rys. 7.3)



Rysunek 7.3: Strona publicznie udostępnionych materiałów wszystkich użytkowników

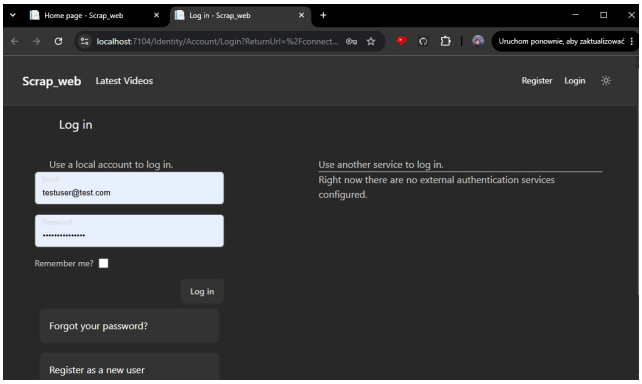
7.4. Transfer przechwyconych plików

Użytkownik po naciśnięciu przycisku przesłania plików, przenoszony jest do domyślnie ustawionej przeglądarki celem rejestracji/logowania. (rys. 7.4)

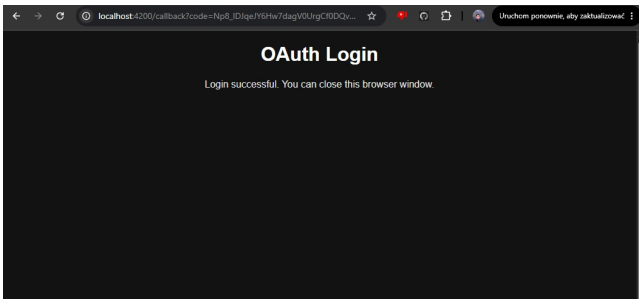
Po prawidłowym procesie weryfikacji, użytkownik otrzymuje informację zwrotną. (rys. 7.5)

W aplikacji desktopowej pojawia się widok umożliwiający wybór pliku, ustawienie tytułu oraz opisu oraz wybór metody transferu danych. (rys. 7.6)

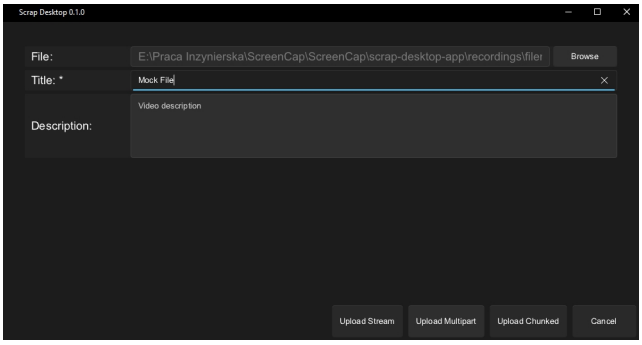
W przypadku powodzenia/niepowodzenia transferu, użytkownik dostaje informację zwrotną(rys. 7.7)



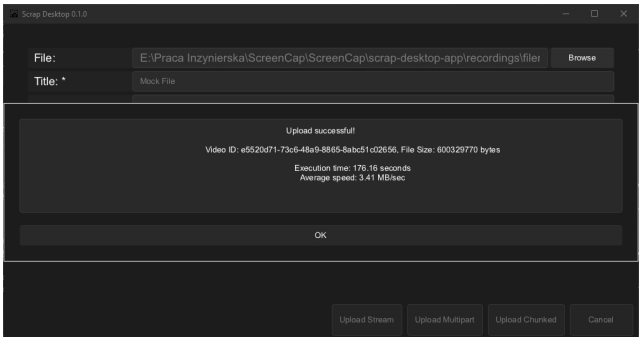
Rysunek 7.4: Widok ekranu logowania



Rysunek 7.5: Widok ekranu potwierdzającego prawidłowe uwierzytelnienie

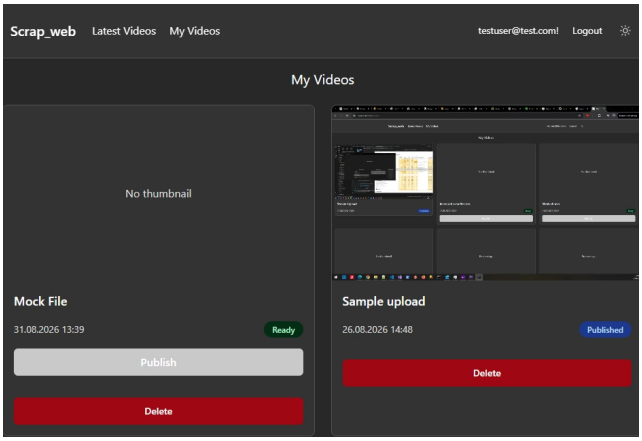


Rysunek 7.6: Widok ekranu wyboru pliku oraz metody transferu



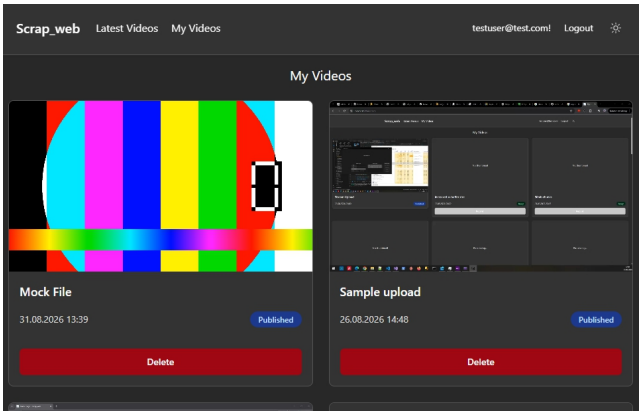
Rysunek 7.7: Widok ekranu potwierdzającego zakończenie transferu

Po stronie aplikacji webowej, użytkownik ma dostęp do wszystkich przesłanych przez siebie plików.(rys. 7.8)



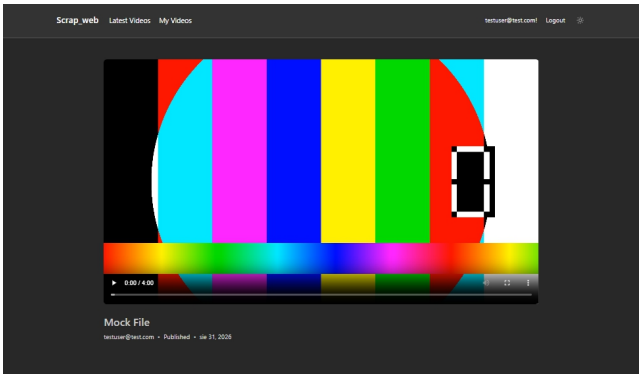
Rysunek 7.8: Widok ekranu wszystkich wideo użytkownika po przesłaniu pliku

Użytkownik ma możliwość udostępnienia publicznie przesłanych plików (rys. 7.9)



Rysunek 7.9: Widok ekranu wszystkich wideo użytkownika po udostępnieniu ostatniego przesłanego pliku

Użytkownik ma możliwość odtworzenia publicznych treści z poziomu przeglądarki. (rys. 7.10)



Rysunek 7.10: Widok strony publicznego wideo

Rozdział 8.

Podsumowanie i dalszy rozwój projektu

W ramach pracy opracowano i zaimplementowano minimalną funkcjonalną wersję systemu integrującego aplikację desktopową odpowiedzialną za przechwytywanie obrazu i dźwięku z aplikacją webową służącą do przesyłania, przechowywania i udostępniania nagrań.

Szczególnym elementem implementacji było opracowanie współbieżnego potoku przetwarzania obrazu i dźwięku. Pozwoliło to przeanalizować praktyczne problemy związane z przechwytywaniem danych w czasie rzeczywistym, ich buforowaniem, enkodowaniem oraz synchronizacją wielu niezależnie przetwarzanych strumieni danych. Implementacja wykazała również ograniczenia wynikające z programowego enkodowania obrazu, w szczególności przy zwiększaniu rozdzielczości i częstotliwości klatek.

Po stronie webowej zaimplementowano podstawowy mechanizm uwierzytelniania, komunikację z aplikacją desktopową, przesyłanie plików oraz ich przechowywanie i prezentację. Mechanizmy przetwarzania materiałów wideo zostały przygotowane w postaci odpowiednich abstrakcji, jednak ich obecna implementacja ma charakter demonstracyjny i stanowi punkt wyjścia do dalszego rozwoju.

Opracowane rozwiązanie należy zatem traktować jako minimalną funkcjonalną wersję systemu i zalążek docelowego rozwiązania, a nie jako kompletną implementację wszystkich planowanych funkcjonalności. Przyjęta architektura umożliwia dalszą rozbudowę, między innymi o rzeczywiste przetwarzanie materiałów wideo po stronie serwera, generowanie metadanych i miniaturek, bardziej rozbudowaną obsługę transferu oraz wykorzystanie sprzętowego enkodowania obrazu.

8.1. Kierunki dalszego rozwoju

- Implementacja testów jednostkowych oraz integracyjnych

- Naprawienie desynchronizacji ścieżek audio
- Integracja akceleratorów sprzętowych do procesu przetwarzania danych.
- Dodanie informacji zwrotnej po stronie ui z danymi dotyczącymi aktualnie przechwytywanego elementu.
- Adaptacyjne ograniczenia FPS
- Implementacja live streamingu
- Integracja FFmpeg po stronie serwera i implementacja faktycznego przetwarzania przesłanych plików, w tym analiza metadanych
- Implementacja właściwej logiki walidacji przesłanego pliku po stronie serwera
- Wznawianie przerwanego transferu pliku po stronie aplikacji desktopowej
- Uzupełnienie implementacji mechanizmów uwierzytelniania dostarczonych przez ASP.NET Identity
- Integracja Azure Blob Storage po stronie serwera
- Implementacja podstawowych funkcjonalności aplikacji webowej, takich jak wyszukiwanie, sortowanie, edycja itp.

Bibliografia

[1] *Microsoft Teams - strona domowa*
`https://teams.live.com/`

(2026.31.08)

[2] *Zoom - strona domowa*
`https://www.zoom.com/`
(2026.31.08)

[3] *Google Meet - strona domowa*
`https://workspace.google.com/intl/pl/resources/video-conferencing/`
(2026.31.08)

[4] *Cisco Webex - strona domowa*
`https://www.webex.com/`
(2026.31.08)

[5] *Discord - strona domowa*
`https://discord.com/`
(2026.31.08)

[6] *Proton Meet - strona domowa*
`https://proton.me/meet`
(2026.31.08)

[7] *Loom - strona domowa*
`https://www.loom.com/`
(2026.31.08)

[8] *OBS Studio - strona domowa*
`https://obsproject.com/`
(2026.31.08)

[9] *vMix - strona domowa*
`https://www.vmix.com/`
(2026.31.08)

[10] *Meld - strona domowa*
https://meldstudio.co/
(2026.31.08)

[11] *BandiCam - strona domowa*
https://www.bandicam.com/
(2026.31.08)

[12] *Media Capture and Streams API (Media Stream) - dokumentacja MDN Web*
https://developer.mozilla.org/en-US/docs/Web/API/Media_Capture_and_Streams_API
(2026.31.08)

[13] *Screen Capture API - dokumentacja MDN Web*
https://developer.mozilla.org/en-US/docs/Web/API/Screen_Capture_API
(2026.31.08)

[14] *Binary Large Object - dokumentacja oracle*
https://docs.oracle.com/cd/E17276_01/html/api_reference/C/blob.html
(2026.31.08)

[15] *YouTube - strona domowa*
https://www.youtube.com/
(2026.31.08)

[16] *Vimeo - strona domowa*
https://vimeo.com/
(2026.31.08)

[17] *Dailymotion - strona domowa*
https://www.dailymotion.com/
(2026.31.08)

[18] *OBS Studio - Github Repository*
https://github.com/obsproject/obs-studio
(2026.31.08)

[19] *User story*
https://en.wikipedia.org/wiki/User_story
(2026.31.08)

[20] *Desmond Gardeslen – Functional Requirements Explained*
https://www.eclipsesuite.com/functional-requirements-explained/
(2026.31.08)

[21] *Hossein Nejati Javaremi:*
Understanding Non-Functional Requirements: The Hidden Drivers of Architecture
`https://hosseinnejati.medium.com/understanding-non-functional-requirements-the-hidden-drivers-of-architecture-855cc30a202b`
(2026.31.08)

[22] *Kostina Prifti, Jessica Morley, Claudio Novelli, Luciano Floridi:*
Regulation by Design: Features, Practices, Limitations, and Governance Implications
`https://www.researchgate.net/publication/380665586_Regulation_by_Design_Features_Practices_Limitations_and_Governance_Implications`
(2026.31.08)

[23] *Azure Blob Storage*
`https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction`
(2026.31.08)

[24] *C4 model – System Context Diagram*
`https://c4model.com/diagrams/system-context`
(2026.31.08)

[25] *C4 model – Container diagram*
`https://c4model.com/diagrams/container`
(2026.31.08)

[26] *C4 model – Component diagram*
`https://c4model.com/diagrams/component`
(2026.31.08)

[27] *Data Model* `https://en.wikipedia.org/wiki/Data_model`
(2026.31.08)

[28] *Rahul Bhandari:*
.NET 8 and .NET 9 will reach End of Support on November 10, 2026
`https://devblogs.microsoft.com/dotnet/dotnet-8-9-end-of-support/`
(2026.31.08)

[29] *FFmpeg 8.0 "Huffman strona domowa*
`https://ffmpeg.org/download.html#release_8.0`
(2026.31.08)

[30] *C language documentation*
`https://learn.microsoft.com/en-us/dotnet/csharp/`
(2026.31.08)

[31] *.NET 8.0 - overview*
https://learn.microsoft.com/pl-pl/dotnet/core/whats-new/dotnet-8/overview
(2026.31.08)

[32] *ASP.NET Core*
https://dotnet.microsoft.com/en-us/apps/aspnet
(2026.31.08)

[33] *Razor Pages*
https://learn.microsoft.com/pl-pl/aspnet/core/razor-pages/
(2026.31.08)

[34] *Entity Framework Core*
https://learn.microsoft.com/en-us/ef/core/
(2026.31.08)

[35] *Microsoft SQL Server*
https://www.microsoft.com/pl-pl/sql-server
(2026.31.08)

[36] *Wprowadzenie do Identity w ASP.NET Core*
https://learn.microsoft.com/pl-pl/aspnet/core/security/authentication/identity?view=aspnetcore-10.0&tabs=visual-studio
(2026.31.08)

[37] *OpenIddict*
https://openiddict.com/
(2026.31.08)

[38] *Tailwind CSS*
https://tailwindcss.com/
(2026.31.08)

[39] *Rust Programming Language*
https://rust-lang.org/
(2026.31.08)

[40] *Cargo*
https://doc.rust-lang.org/book/ch01-03-hello-cargo.html
(2026.31.08)

[41] *Slint*
https://slint.dev/
(2026.31.08)

[42] *CapSoftware scap*
https://crates.io/crates/scap/0.1.0-beta.1
(2026.31.08)

[43] *CPAL - Cross-Platform Audio Library*
`https://crates.io/crates/cpal`
(2026.31.08)

[44] *ffmpeg-next*
`https://crates.io/crates/ffmpeg-next/8.0.0`
FFmpeg
`https://www.ffmpeg.org/`
(2026.31.08)

[45] *Rubato*
`https://crates.io/crates/rubato`
(2026.31.08)

[46] *ringbuf*
`https://crates.io/crates/ringbuf`
(2026.31.08)

[47] *crossbeam-channel*
`https://crates.io/crates/crossbeam-channel`
(2026.31.08)

[48] *tokio*
`https://crates.io/crates/tokio`
(2026.31.08)

[49] *reqwest*
`https://crates.io/crates/reqwest`
(2026.31.08)

[50] *serde*
`https://crates.io/crates/serde`
serde_json
`https://crates.io/crates/serde_json`
(2026.31.08)

[51] *uuid*
`https://crates.io/crates/uuid`
(2026.31.08)

[52] *Presentation timestamp (PTS) - defini*
`https://wiki.svta.org/presentation-timestamp/`
(2026.31.08)